MinRV32: A Hands-On, Open-Source RV32I Pipeline for Teaching Digital Design from RTL to FPGA

Project documentation (compiled from `docs/`)

October 11, 2026

Abstract

MinRV32 is an educational RV32I processor implemented as a clean, modular, five-stage pipeline (F/D/E/M/W) in VHDL. The design is intentionally small but complete: it includes forwarding, stall/flush hazard handling, a Harvard instruction/data memory split, a linker-aware memory map, and a practical build flow that uses a RISC-V GCC toolchain to generate hex images for IMEM/DMEM. This document consolidates the repository documentation into a single, coursework-friendly overview and quickstart.

Contents

1 Project showcase: an educational route from RTL to FPGA **4**
- 1.1 Step 1: read/modify VHDL . . . 4
- 1.2 Step 2: inspect a module's schematic (before simulating) . . . 5
- 1.3 Step 3: simulate modules with unit testbenches . . . 7
- 1.4 Step 4: run assembly tests (correctness via signature + optional traces/waves) . . . 8
- 1.5 Step 5: run freestanding C tests (same signature mechanism) . . . 9
- 1.6 Step 6: explore the design interactively in DigitalJS . . . 11
- 1.7 Step 7: implement on FPGA and validate with an ILA capture . . . 11

2 What this project is for **13**
- 2.1 What you learn by studying or modifying this core . . . 13
- 2.2 Textbook lineage . . . 13

3 Microarchitecture overview **14**
- 3.1 One detail that matters on FPGA: synchronous memories . . . 16
- 3.2 Stage-by-stage summary . . . 16
- 3.3 Major RTL blocks (module map) . . . 19
- 3.4 Hazard unit operation . . . 20
  - 3.4.1 Forwarding-enabled mode . . . 20
  - 3.4.2 Stall-only mode . . . 20
  - 3.4.3 Shared stall and flush outputs . . . 21

4 ISA coverage and control signals **22**
- 4.1 Instruction groups . . . 22
- 4.2 Control signal glossary (selected) . . . 22
- 4.3 Control naming and stage suffixes . . . 22
- 4.4 Compact control matrix (from `docs/instruction_control.md`) . . . 22

5 Memory map, linker script, and GCC-to-hex flow **24**
5.1 Harvard memory map . . . 24
5.2 Why these addresses? . . . 24
5.3 A note on address decoding in the simple memories . . . 24
5.4 Data-side reads from the text region (`.rodata`) . . . 24
5.5 Linker script (`link.ld`) . . . 24
5.6 GCC → ELF → hex images . . . 25
5.7 Future work: one memory map for the core and Spike . . . 25

6 Directed instruction tests and helper scripts **27**
6.1 ISA tests . . . 27
6.2 Prerequisites . . . 27
6.3 One-test and batch workflows . . . 27
6.4 Waveform dumps (GTKWave) . . . 28
6.5 Unit-level testbenches . . . 28
6.6 DigitalJS interactive simulation . . . 28
6.7 Trace output (optional) . . . 30
6.8 Authoring new tests . . . 30
6.9 Golden signatures with Spike . . . 30
6.10 Running a program on Spike by hand . . . 32
6.11 Stepping through a program in Spike's debugger . . . 32
6.12 Script roles (high level) . . . 33

7 Vivado/XSim and FPGA bring-up **35**
7.1 Vivado simulation (XSim) checklist . . . 35
7.2 Vivado source organization tip (exact lists) . . . 36
7.3 VHDL-2008 enable (Tcl snippet) . . . 38
7.4 Pointing the testbench at hex/signature files . . . 38
7.5 Optional: per-instruction trace wrappers . . . 38
7.6 Expected warnings . . . 38
7.7 BRAM initialization: simulation vs synthesis . . . 39
7.8 Synthesis top generics (init files and sizes) . . . 39
7.9 On-chip debugging (ILA) . . . 39
7.10 Validate a program via the signature window (ILA capture) . . . 39
7.11 Implementation results . . . 40

8 Deviations from Harris & Harris **42**
8.1 Summary table . . . 42
8.2 Hazard-unit design notes (practical guards) . . . 43
8.3 True dual-port IMEM for `.rodata` loads . . . 43
8.4 Register file: rising-edge write with bypass . . . 44
8.5 Textbook vs BRAM-style timing at the stage boundary . . . 44
8.6 Why the synchronous-memory change matters . . . 44

9 Quickstart (common flows) **45**
9.1 Run a single test (GHDL) . . . 45
9.2 Run a single test with trace and waves . . . 45
9.3 Run all tests (regression) . . . 45

9.4 Generate or check golden signatures (Spike) . . . 45
9.5 Run a unit testbench (module bring-up) . . . 45
9.6 Build hex images for Vivado/BRAM init . . . 45
9.7 Interactive schematic + simulation (DigitalJS, simple module) . . . 45
9.8 Interactive full-pipeline simulation with firmware loaded . . . 46
9.9 Typical artifacts . . . 46

References **47**

1 Project showcase: an educational route from RTL to FPGA

This section presents the project as a sequence of short, reproducible labs. Each step produces something you can inspect: waveforms, traces, signatures, or on-board memory state. Figure 1 gives the overall route from editing the RTL to validating on the FPGA.

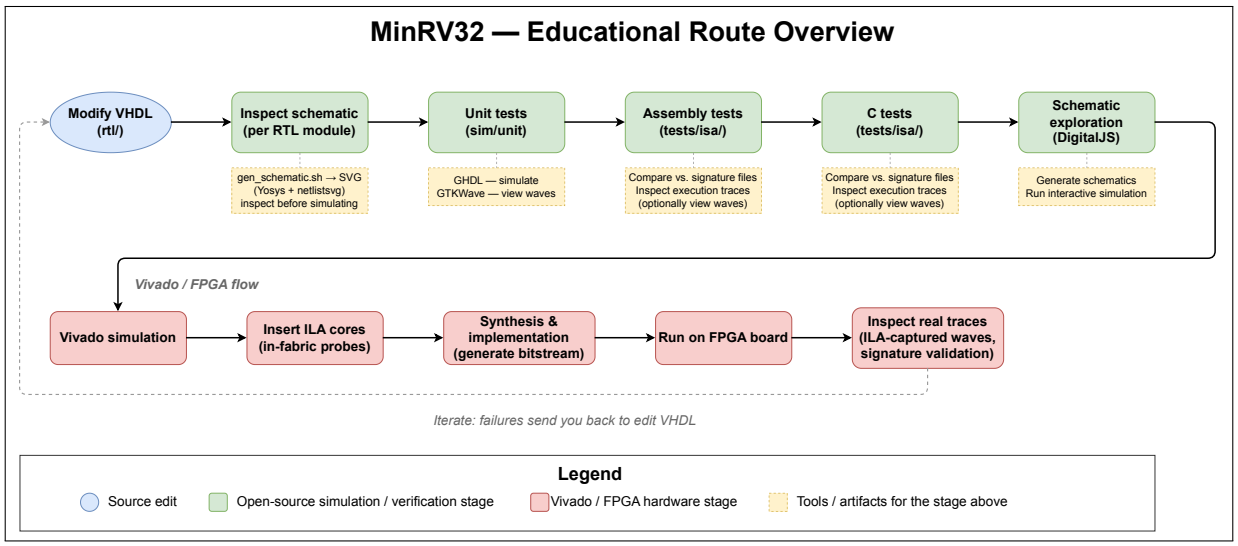


Figure 1: Educational route overview. After editing the RTL, a module's schematic can be generated for inspection (Yosys + `netlistsvg`) before any simulation is run. After simulation, the design can be implemented and validated on the FPGA with an ILA capture.

1.1 Step 1: read/modify VHDL

The RTL is modular (ALU, regfile, hazard unit, memory access helper, etc.), so a change is usually localized to one small, readable entity.

Example VHDL: ALU combinational process (complete case statement)

```
architecture rtl of alu is
begin
  -- Purpose: Execute-stage ALU (combinational). Computes the result for the
  -- selected ALUControl_t operation.
  p_alu_comb : process(op, a, b)
    variable alu_res : word_t;
  begin
    alu_res := (others => '0');
    case op is
      when ALU_ADD =>
        -- Signed addition handles ADD/ADDI/AUIPC path.
        alu_res := std_logic_vector(signed(a) + signed(b));
      when ALU_SUB =>
        -- Signed subtraction handles SUB instruction.
        alu_res := std_logic_vector(signed(a) - signed(b));
      when ALU_AND =>
        -- Bitwise logical operations rely on std_logic_vector operators.
        alu_res := a and b;
```

```
      when ALU_OR =>
        alu_res := a or b;
      when ALU_XOR =>
        alu_res := a xor b;
      when ALU_SLT =>
        -- Signed comparison returns bit0=1 when RS1 < RS2.
        if signed(a) < signed(b) then
          alu_res(0) := '1';
        end if;
      when ALU_SLTU =>
        -- Unsigned compare variant for SLTU.
        if unsigned(a) < unsigned(b) then
          alu_res(0) := '1';
        end if;
      when ALU_SLL =>
        -- Logical left shift; shift amount from B[4:0].
        alu_res := std_logic_vector(shift_left(unsigned(a),
                     to_integer(unsigned(b(4 downto 0)))));
      when ALU_SRL =>
        -- Logical right shift with zero-fill.
        alu_res := std_logic_vector(shift_right(unsigned(a),
                     to_integer(unsigned(b(4 downto 0)))));
      when ALU_SRA =>
        -- Arithmetic right shift preserves sign bit.
        alu_res := std_logic_vector(shift_right(signed(a),
                     to_integer(unsigned(b(4 downto 0)))));
    end case;
    result <= alu_res;
  end process p_alu_comb;
end architecture rtl;
```

1.2 Step 2: inspect a module's schematic (before simulating)

Before running any simulation, the structure of an edited module can be checked visually. `scripts/gen_schematic.sh` synthesizes an entity with Yosys (via its GHDL front-end) and renders an SVG schematic with `netlistsvg`, so design mistakes are often caught just by looking at the elaborated logic.

```
./scripts/gen_schematic.sh --list           # list discoverable RTL entities
./scripts/gen_schematic.sh alu              # hierarchical schematic of one module
./scripts/gen_schematic.sh --flat 1 riscvpipe  # expand one level of hierarchy
```

The SVG lands under `build/schematics/<entity>.svg` (Figure 2 shows the result for the `alu` entity). The same tool underlies the microarchitecture block diagrams in this manual and the DigitalJS export described in Step 6 (Section 6.6); see that section for the prerequisite packages.

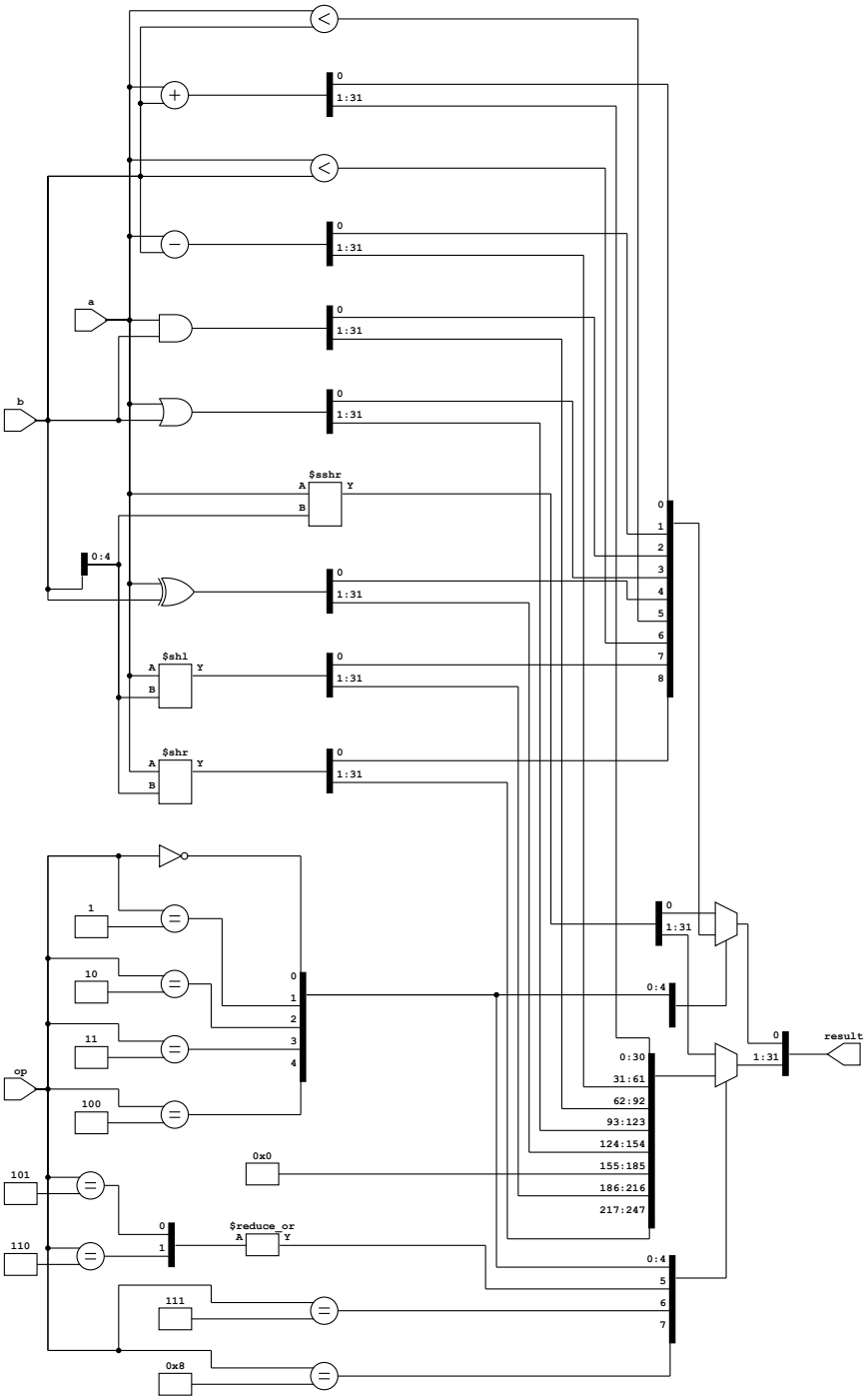


Figure 2: Elaborated schematic of the `alu` entity rendered by `gen_schematic.sh` (Yosys + `netlistsvg`). The `op` decoder (`$eq` comparators, left of centre) selects, through the output multiplexers, one of the candidate results computed by the arithmetic, shift, and comparison operators. By default the script drops the multiplexers' don't-care default input for the unused `op` encodings (`opt_expr -mux_undef`), so no floating stubs clutter the diagram; pass `-keep-undef` to keep them.

1.3 Step 3: simulate modules with unit testbenches

For component bring-up, the repository includes small self-checking unit testbenches under `sim/unit/`.

Example unit testbench: checking the ALU (complete architecture body)

```
begin
  dut : entity work.alu
    port map (
      op     => op,
      a      => a,
      b      => b,
      result => result
    );

  stim : process
  begin
    check(op, a, b, result, ALU_ADD,  x"00000000", x"00000000", "add zero");
    check(op, a, b, result, ALU_ADD,  x"00000001", x"00000002", "add small");
    check(op, a, b, result, ALU_SUB,  x"00000003", x"00000001", "sub small");
    check(op, a, b, result, ALU_AND,  x"0f0f0f0f", x"00ff00ff", "and");
    check(op, a, b, result, ALU_OR,   x"0f0f0f0f", x"00ff00ff", "or");
    check(op, a, b, result, ALU_XOR,  x"0f0f0f0f", x"00ff00ff", "xor");
    check(op, a, b, result, ALU_SLT,  x"ffffffff", x"00000001", "slt -1 < 1");
    check(op, a, b, result, ALU_SLTU, x"00000001", x"ffffffff", "sltu 1 < 0xffffffff");

    check(op, a, b, result, ALU_SLL,  x"00000001", x"0000001f", "sll 31");
    check(op, a, b, result, ALU_SRL,  x"80000000", x"0000001f", "srl 31");
    check(op, a, b, result, ALU_SRA,  x"80000000", x"0000001f", "sra 31");

    report "tb_alu: PASS" severity note;
    finish;
  end process;
end architecture sim;
```

Run + dump waves

```
./scripts/run_unit_tb.sh --fst alu
./scripts/view_wave.sh build/waves/tb_alu.fst
```

GTKWave then shows the unit-test waveform (Figure 3).

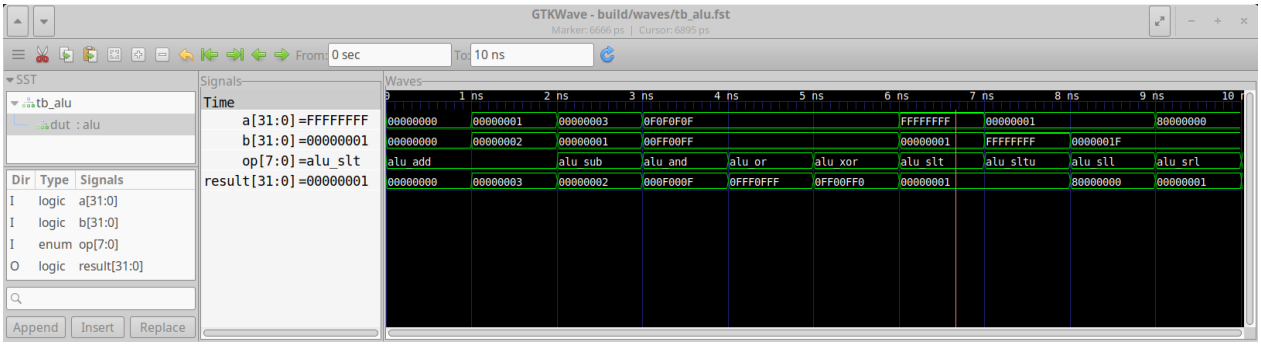


Figure 3: Unit-test waveform.

1.4 Step 4: run assembly tests (correctness via signature + optional traces/waves)

Integration tests live under `tests/isa/`. Each test writes a signature window into DMEM at `0x20000100`. The expected values are stored in `tests/isa/<name>.sig`, and the testbench compares observed vs expected.

Example assembly test: write 6 signature words

```
.section .text
.globl _start
.equ SIGNATURE_BASE, 0x20000100

_start:
    li t0, SIGNATURE_BASE

    li t1, 5
    addi t2, t1, -3
    sw t2, 0(t0)

    li t1, -5
    addi t2, t1, 6
    sw t2, 4(t0)

    li t1, 0x7ff
    addi t2, t1, 1
    sw t2, 8(t0)

    li t1, -2048
    addi t2, t1, -1
    sw t2, 12(t0)

    li t1, 0
    addi t2, t1, 0
    sw t2, 16(t0)

    addi t2, zero, -123
    sw t2, 20(t0)

end_loop:
```

```
    beq zero, zero, end_loop
```

Run + view trace and waves

```
./scripts/run_test.sh --trace addi_only
./scripts/run_test.sh --fst addi_only
./scripts/view_wave.sh build/waves/addi_only.fst
```

Per-instruction trace: decoded ASM + stage control sample output

```
ASM=unknown
FETCH: PC=0x00000000 INSTR=0xXXXXXXXX
DECODE [addi x0, x0, 0]: ImmSrc=I
EXEC [---]: SrcA=rs1 ALUSrc=rs2 ALUControl=add
MEM [---]:
WB [---]: ResultSrc=alu

ASM=lui x5, 0x20000000
FETCH: PC=0x00000004 INSTR=0x200002B7
DECODE [lui x5, 0x20000000]: ImmSrc=U
EXEC [addi x0, x0, 0]: SrcA=rs1 ALUSrc=imm ALUControl=add
MEM [---]:
WB [---]: ResultSrc=alu

ASM=addi x5, x5, 256
FETCH: PC=0x00000008 INSTR=0x10028293
DECODE [addi x5, x5, 256]: ImmSrc=I
EXEC [lui x5, 0x20000000]: SrcA=zero ALUSrc=imm ALUControl=add
MEM [addi x0, x0, 0]:
WB [---]: ResultSrc=alu

ASM=addi x6, x0, 5
FETCH: PC=0x0000000C INSTR=0x00500313
DECODE [addi x6, x0, 5]: ImmSrc=I
EXEC [addi x5, x5, 256]: SrcA=rs1 ALUSrc=imm ALUControl=add
MEM [lui x5, 0x20000000]:
WB [addi x0, x0, 0]: ResultSrc=alu RegWrite=yes
HAZARD [addi x5, x5, 256]: StallF=run StallD=run FlushD=keep FlushE=keep FwdA=M FwdB=rf
```

1.5 Step 5: run freestanding C tests (same signature mechanism)

Freestanding C tests follow the same rule: compute deterministic values, then store them into the signature window at `0x20000100`.

Example C: tests/isa/factorial.c

```
#include <stdint.h>

#define SIGNATURE_BASE ((volatile uint32_t *)0x20000100)

__asm__ (
  ".section .init\n"
  ".globl _start\n"
  "_start:\n"
```

```
  "  la sp, __stack_top\n"
  "  jal ra, main\n"
  "1: j 1b\n"
);

static uint32_t mul_u32(uint32_t a, uint32_t b);

static uint32_t factorial(uint32_t n);

int main(void)
{
  volatile uint32_t *sig = SIGNATURE_BASE;

  uint32_t n = 6u;
  uint32_t res = 0;

  res = factorial(n);

  sig[0] = res;

  while (1) {
  }
}
```

Run + view trace and waves

```
./scripts/run_test.sh --trace factorial
./scripts/run_test.sh --fst factorial
./scripts/view_wave.sh build/waves/factorial.fst
```

Example test output snippet without trace

```
Artifacts written to /home/pmousoul/repos/MinRV32-SC/build:
  - factorial.elf       : full ELF
  - factorial.text.hex  : load into instruction memory (text/rodata)
  - factorial.data.hex  : load into data memory (.data, zero-extend for .bss)
  - factorial.S          : GCC-generated assembly from the C source

Sizes:
  - text/rodata: 132 bytes (33 words)  [IMEM capacity: 8192 bytes]
  - data/sdata : 0 bytes (0 words)  [DMEM capacity: 8192 bytes]

/home/pmousoul/repos/MinRV32-SC/sim/tb_minrv32.vhd:713:9:@3425ns:(report note): TEST
    PASSED
simulation stopped @3425ns with status 0
```

The resulting execution waveform for computing 6! is shown in Figure 4.



Figure 4: Factorial C program execution waveform for calculating 6!

1.6 Step 6: explore the design interactively in DigitalJS

The same netlist that produces the static schematics of Step 2 can be exported to DigitalJS for interactive, in-browser simulation: drive inputs by clicking and watch values propagate through the gates. With real firmware pre-loaded it even single-steps the whole pipeline. This is a visual complement to GTKWave (for leaf modules) and to the full-core tests (for the whole pipeline); it is not signature-checked, so it does not replace the testbench pass/fail. See Section 6.6 for the commands and the firmware-loaded export. Although the simulation is not signature-checked, it is a powerful tool for visually confirming the design's behaviour on the FPGA, where visibility is limited to the ILA capture points. Additionally, the signature values written in the DMEM, usually at the end of the executed program, can be manually verified against the expected values in the corresponding `.sig` file.

1.7 Step 7: implement on FPGA and validate with an ILA capture

After simulation passes, Vivado can synthesize and implement the same RTL. On hardware, you validate results with an Integrated Logic Analyzer (ILA) triggered on stores into the signature window – the same `.sig` words that simulation already checks (Figure 5).

Content of the tests/isa/full_core.sig file

```
00004a78
eca8643f
c77fc03f
0000018e
0002018e
0001827e
c7028a47
000001cb
fd23373a
6cf9b359
```

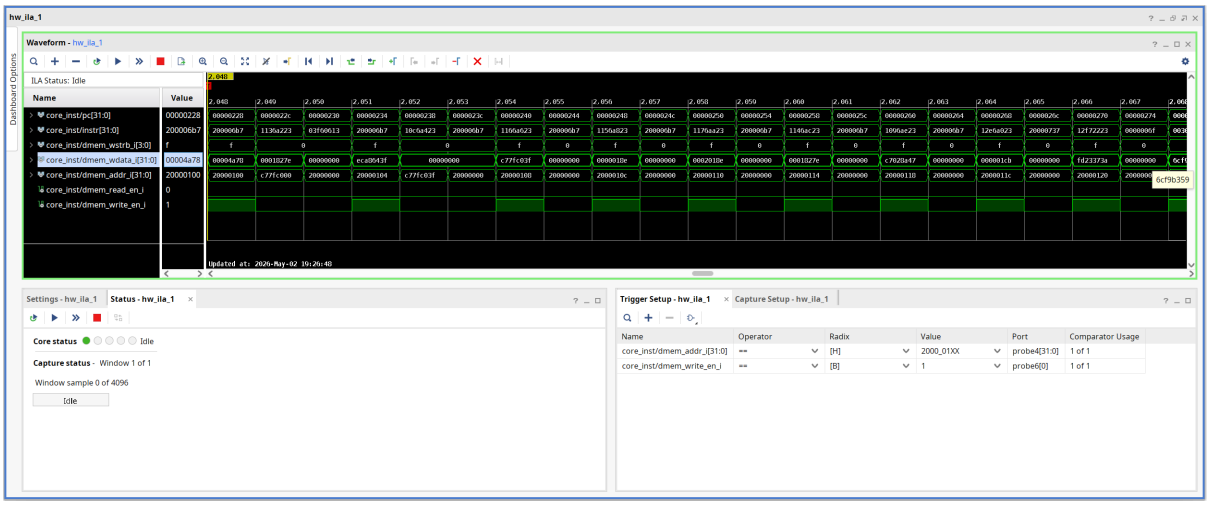


Figure 5: Hardware Manager + ILA waveform capturing signature stores of the full_core.c program running on MinRV32 on the FPGA.

2 What this project is for

MinRV32 is a teaching-oriented implementation of an RV32I pipelined CPU. Its goal is to help you learn how a modern in-order pipeline is structured and validated in an HDL, and how the resulting RTL maps onto FPGA-friendly building blocks (especially synchronous Block RAMs).

2.1 What you learn by studying or modifying this core

- How a five-stage pipeline is partitioned (Fetch/Decode/Execute/Memory/Writeback) and how stage registers define timing.
- How a hazard unit combines forwarding, stalling, and flushing to preserve correctness.
- How a Harvard memory map and a custom linker script make bare-metal C programs *actually runnable* on the hardware model.
- How to validate ISA behavior with directed tests and self-checking signatures.
- HDL coding practice: writing synthesizable RTL (design for synthesis) and writing testbenches for design verification.
- Using open-source tools to simulate, verify, and visualize HDL designs (e.g. GHDL, GTKWave, DigitalJS).
- How to simulate and debug in Xilinx Vivado (XSim), and how to instrument an FPGA build with ILA.

2.2 Textbook lineage

The RTL follows the structure and naming conventions of the Harris & Harris pipelined reference design (stage-suffixed signals such as `PCF`, `InstrD`, `ALUControlE`, `MemWriteM`, `ResultSrcW`). Section 8 summarizes the intentional deviations made to support a synthesis-ready, BRAM-friendly implementation and a slightly richer RV32I subset.

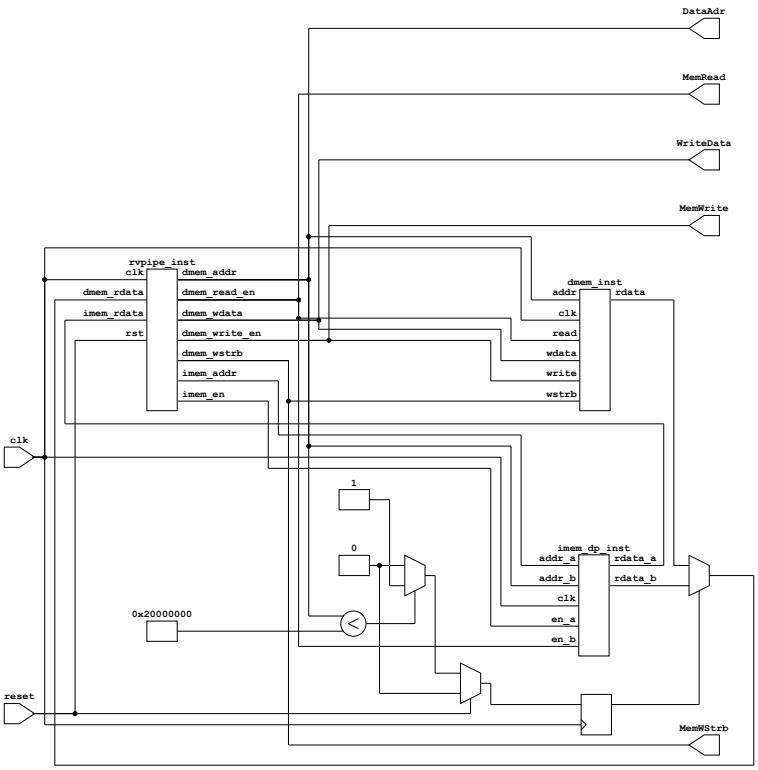


Figure 6: System level (`top`). The synthesis top wires the pipelined core (`rvpipe_inst`) to a dual-port instruction memory (`imem_dp_inst`) and a data memory (`dmem_inst`); the dual-port IMEM also serves `.rodata` reads on port B. Auto-generated with `netlistsvg`.

3 Microarchitecture overview

The core is a five-stage RV32I pipeline (F/D/E/M/W). Instructions overlap in flight; stage boundaries are defined by pipeline registers, and a dedicated hazard unit generates forwarding, stall, and flush control. The block schematics in this section (Figures 6–9) are generated directly from the RTL with Yosys and `netlistsvg` (via `scripts/gen_schematic.sh`) in hierarchical mode, so every labelled box is a real sub-module instance. They progress from the system boundary (Figure 6), through the core (Figure 7) and the five datapath stages (Figure 8), down to one stage's internals (Figure 9).

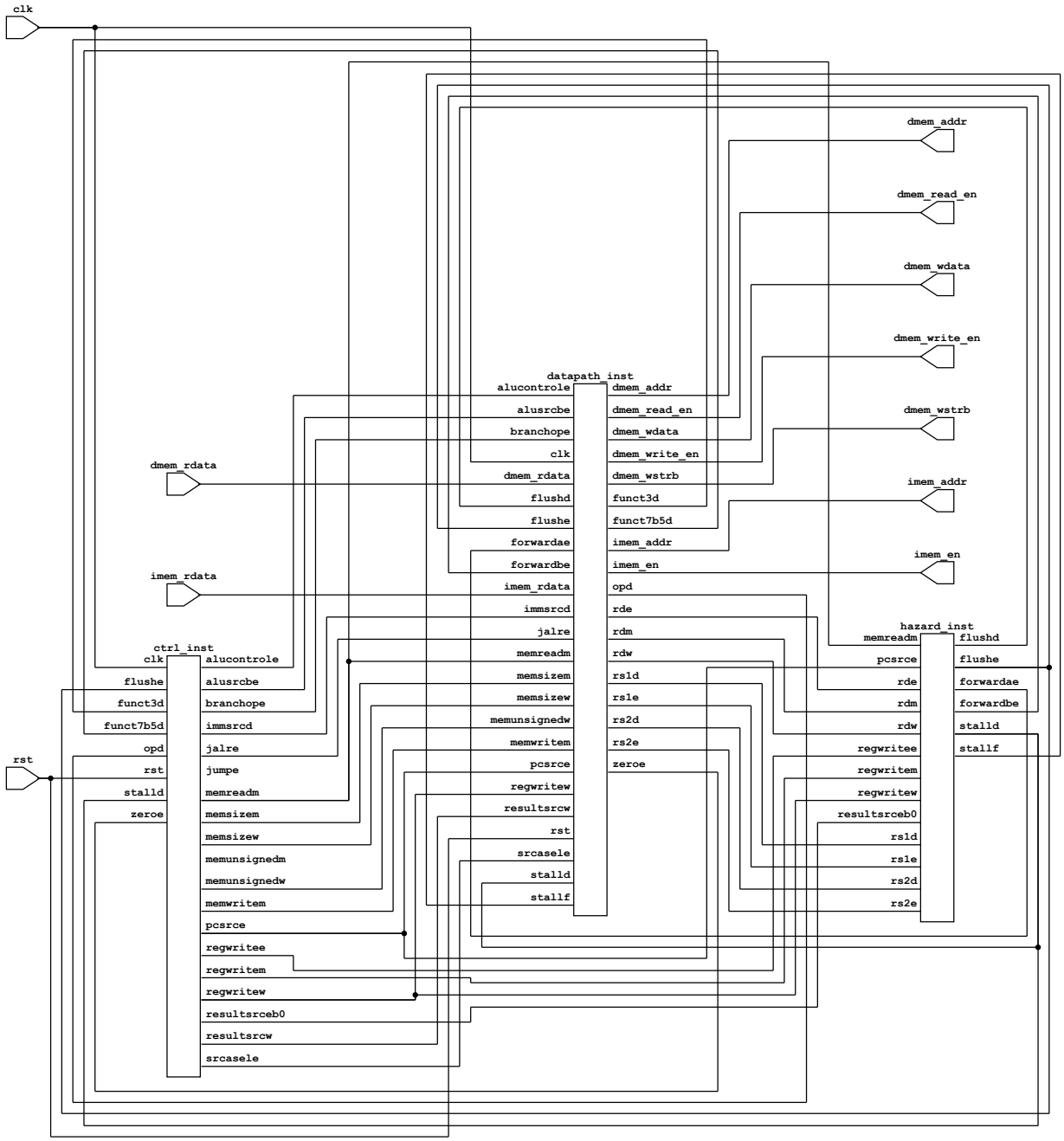


Figure 7: Core level (`riscvpipe`). The pipelined core binds the controller (`ctrl_inst`), the structural datapath (`datapath_inst`), and the hazard unit (`hazard_inst`); the control and hazard nets between them are visible as the inter-block wiring. Auto-generated with `netlistsvg`.

3.1 One detail that matters on FPGA: synchronous memories

Instruction and data memories are written in a BRAM-friendly style with synchronous reads. As a result, the registered memory outputs naturally align with pipeline stage boundaries: the IMEM output provides `InstrD` and the DMEM output provides `ReadDataW`. A small `flush_d_hold` mechanism inserts a NOP after redirects to implement a textbook-style decode flush even though BRAM outputs cannot be "overwritten" like a conventional F→D register.

3.2 Stage-by-stage summary

Fetch (F) A `PCF` register feeds the instruction memory address. Next-PC selection chooses `PC+4` or an Execute-stage redirect (taken branch/jump or JALR).

Decode (D) The controller decodes opcode/funct fields and the register file reads `rs1`/`rs2`. Immediate expansion produces `ImmExtD`. Decode-stage pipeline state supports stall/flush.

Execute (E) The ALU runs with forwarded operands; a branch unit evaluates all RV32I branch types. Redirect decisions occur here and trigger flush of wrong-path work.

Memory (M) The address and store data are registered; a memory access helper produces byte strobes and aligned store data for SB/SH/SW.

Writeback (W) Load sign/zero extension is applied in W, the writeback mux selects ALU vs memory vs PC+4, and the register file writes on the rising edge with a small combinational bypass to resolve same-cycle W→D read-after-write.

Per-instruction stage walkthrough The repository keeps a stage-by-stage walkthrough for each supported RV32I instruction group in `docs/instruction_control.md`. This PDF references that Markdown source to avoid duplicating the same table-level material in two places.

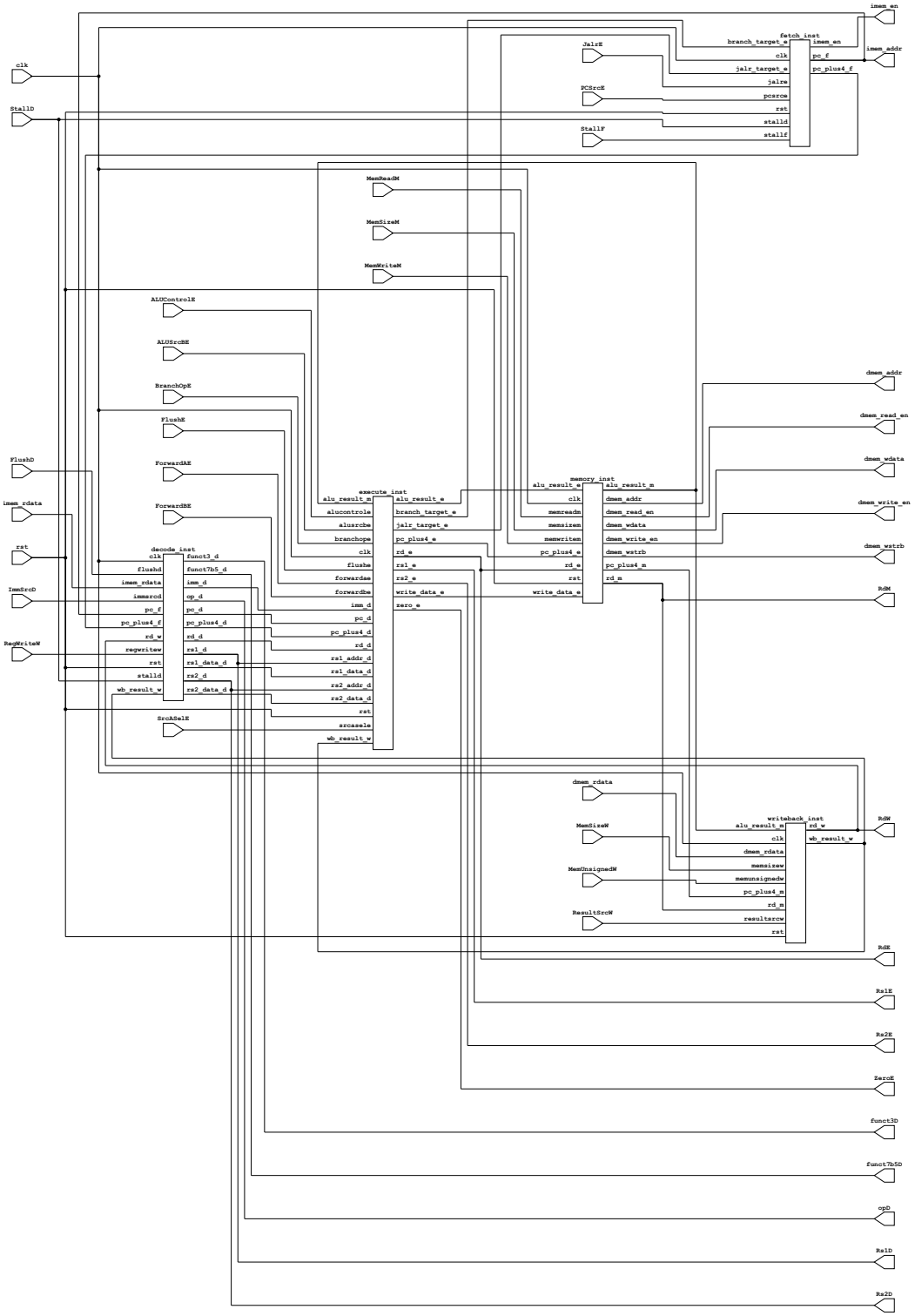


Figure 8: Datapath stages (`datapath_pipe`). The purely structural datapath instantiates the five per-stage wrappers (`fetch_inst`, `decode_inst`, `execute_inst`, `memory_inst`, `writeback_inst`) and wires the inter-stage signals between them. Auto-generated with `netlistsvg`.

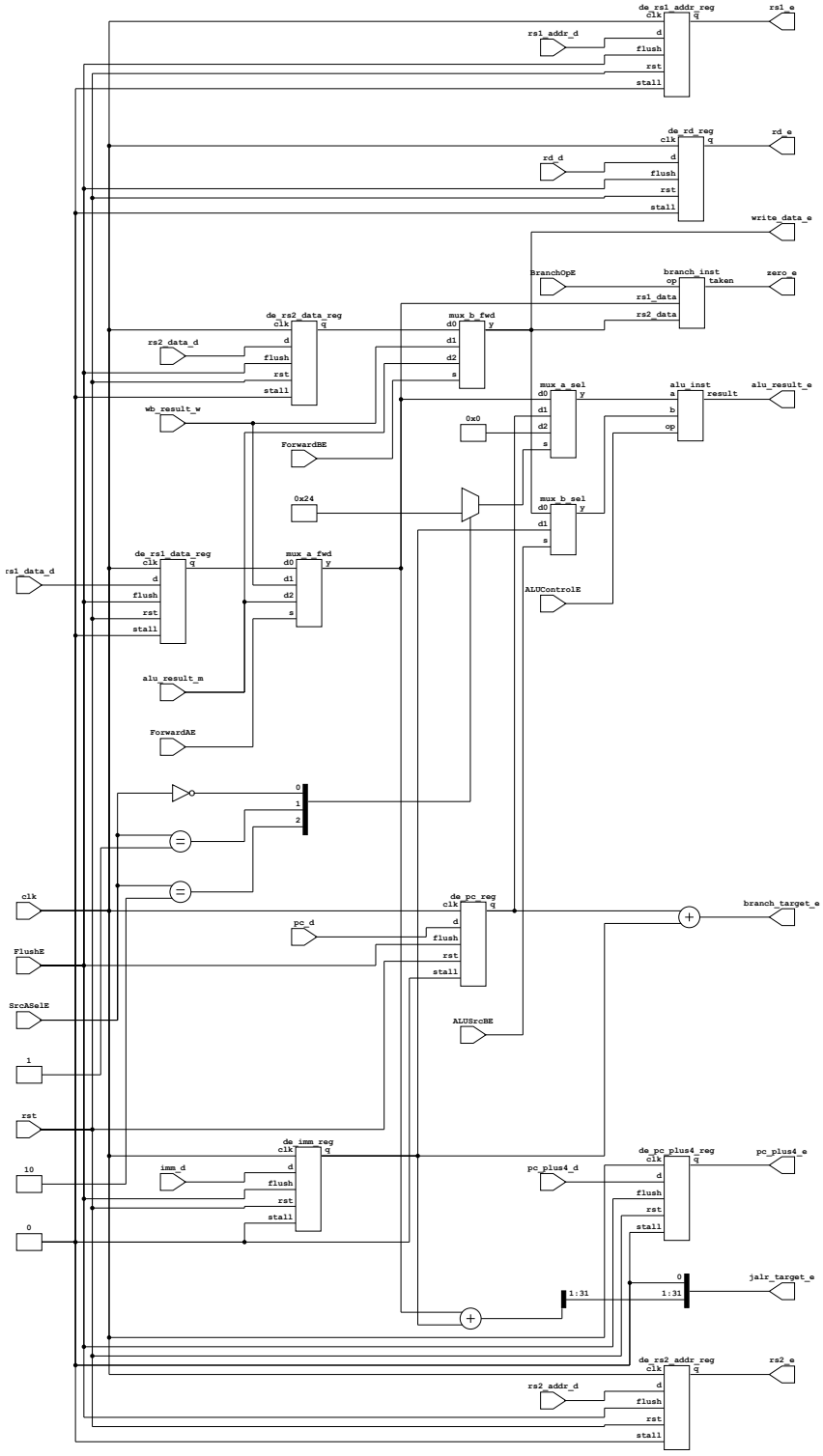


Figure 9: Execute-stage internals (`execute_stage`), showing the named functional units: the ALU (`alu_inst`), the branch unit (`branch_inst`), the forwarding and operand-select muxes (`mux_a_fwd`/`mux_b_fwd`, `mux_a_sel`/`mux_b_sel`), the D→E pipeline registers (`de_*_reg`), and the branch-target adder. Auto-generated with `netlistsvg`.

3.3 Major RTL blocks (module map)

At the core level, `riscvpipe` binds three blocks (Figure 7): the controller, the structural datapath, and the hazard unit.

The design is intentionally modular:

- `riscvpipe.vhd`: binds controller, datapath, and hazard unit into the pipelined core.
- `top.vhd`: wires the core to Harvard IMEM/DMEM models, including `.rodata` reads via dual-port IMEM. Exposes textbook observation ports (`DataAdr`, `WriteData`, `MemWrite`, `MemRead`, `MemWStrb`) so the VHDL testbench can monitor memory traffic directly.
- `fpga_top.vhd`: thin Vivado synthesis wrapper around `top`. Exposes `clk`, `reset`, and a 1-bit `led` activity indicator (the MSB of a free-running blink counter XORed with an XOR reduction of every observation signal; the counter makes the LED blink while the clock runs, and the XOR fold prevents Vivado from eliminating the datapath as dead code). It also makes the core clock from the board clock with an MMCM (the FPGA's clock generator block; by default 80 MHz on Basys3 and ZedBoard). The FPGA pad ring stays at 3 IOs; internal observation nets remain accessible via ILA with `MARK_DEBUG`.
- `controller_pipe.vhd`: decode in D; registers control into E/M/W.
- `datapath_pipe.vhd`: **structural** top-level datapath that instantiates five per-stage wrapper entities and wires inter-stage signals (no inline processes or logic).
- `hazard_unit.vhd`: forwarding, load-use stalls, and control-hazard flush.
- `imem_dp.vhd`, `dmem.vhd`: synchronous-read memories coded to infer FPGA BRAM.
- `mem_access_unit.vhd`: SB/SH/SW byte strobes + store alignment.
- `load_extend_unit.vhd`: combinational load-data sign/zero extension applied in W.

Datapath stage hierarchy `datapath_pipe.vhd` instantiates the following stage wrappers, each containing named entity instances for pipeline registers (`pipe_reg`), muxes, and functional units. This produces a navigable per-stage hierarchy in Vivado's elaborated design view.

`fetch_stage` PC register (`pipe_reg`), PC+4 adder, next-PC muxes, BRAM read-enable.

`decode_stage` F→D pipeline registers, NOP insertion (`flush_d_hold`), `regfile`, `extend`.

`execute_stage` D→E pipeline registers, forwarding `mux3` instances, ALU-B `mux2`, `alu`, `branch_unit`, target computation.

`memory_stage` E→M pipeline registers, `mem_access_unit`.

`writeback_stage` M→W pipeline registers, `load_extend_unit`, WB result `mux3`.

A reusable `pipe_reg` entity (generic `WIDTH`, ports for `clk/rst/flush/stall/d/q`) implements all explicit pipeline registers.

3.4 Hazard unit operation

The hazard unit resolves the three hazard classes of a 5-stage pipeline. A single elaboration-time generic, `FORWARDING_ENABLE`, selects one of two mutually exclusive bodies through an `if ...generate`. The stall and flush *outputs* (`StallF`, `StallD`, `FlushD`, `FlushE`) are produced by shared logic that is identical in both modes; only the internal stall condition (`stallD_i`) and the forwarding muxes differ between them. The goal of this subsection is to explain *why each condition is required*, not merely what it computes.

3.4.1 Forwarding-enabled mode

The operand-forwarding muxes choose, for the instruction in Execute, where each ALU input comes from:

Select	Source	Asserted when
`"00"`	register file	no hazard, or source is `x0` (default)
`"10"`	`ALUResultM`	`RsxE=RdM` and `RegWriteM='1'` and `MemReadM='0'` and `RsxE/=x0`
`"01"`	`ResultW`	`RsxE=RdW` and `RegWriteW='1'` and `RsxE/=x0`

- **M before W priority.** When the same register is in flight in both M and W, the copy in M is the newer write and must win; the `if/elsif` order encodes this.
- **`MemReadM='0'` on the M path.** A load in M holds only the computed *address* in `ALUResultM`, not its data; forwarding it would corrupt the operand. Loads are instead covered by the load-use stall, after which the value forwards safely from W. The W path needs no such guard, because W always carries the final committed value.
- **`RsxE/=x0`.** `x0` is hardwired to zero and is never a real dependency.

The remaining data hazard is the *load-use* case: a load in E (`ResultSrcEb0='1'`) produces its result only at the end of M, too late to forward into the dependent instruction already in E. The unit stalls F and D for one cycle and bubbles E; one cycle later the load is in W and forwards normally:

```
stallD_i = ResultSrcEb0='1'
           and ( (Rs1D=RdE and Rs1D/=x0) or (Rs2D=RdE and Rs2D/=x0) )
```

The `/=x0` guards suppress a false stall when the consumer reads `x0`.

3.4.2 Stall-only mode

For debugging and for demonstrating why forwarding matters, the unit can be elaborated with `FORWARDING_ENABLE=false`; the test scripts expose this. The forwarding muxes are tied to `"00"` (the ALU always reads the register file), so every RAW dependency must be resolved by stalling until the producer writes back in W. `stallD_i` becomes the OR of the load-use term above and a generic RAW term:

```
rawStallD_b = (Rs1D=RdE or Rs2D=RdE) and RegWriteE='1' and RdE/=x0
           or (Rs1D=RdM or Rs2D=RdM) and RegWriteM='1' and RdM/=x0
           or (Rs1D=RdW or Rs2D=RdW) and RegWriteW='1' and RdW/=x0
```

- **`RegWrite*='1'.`** Stores and branches reuse the `Rd` bit-field for immediate bits, so a non-zero `Rd` does not imply a register write. Gating on the real control signal avoids stalling on non-existent dependencies.
- **`Rd*/=x0.`** A write to `x0` is discarded by the ISA, so an instruction whose destination is `x0` is not a producer.

3.4.3 Shared stall and flush outputs

```
StallF = stallD_i and not PCSrcE
StallD = stallD_i and not PCSrcE
FlushD = PCSrcE
FlushE = stallD_i or PCSrcE
```

- `StallF`/`StallD` freeze the PC and the F/D register so the dependent instruction stays in D until the hazard clears.
- `FlushD=PCSrcE` kills the speculatively fetched instruction in D when a branch or jump resolves taken in E.
- `FlushE` bubbles E for either reason: a data stall has frozen D (the instruction in E must not re-execute), or a taken branch must squash its wrong-path successor in E.
- **The `and not PCSrcE` guard.** When a taken branch coincides with a data stall the PC must still redirect. `StallF` is the only gate on the PC update – there is no flush on the PC – so the guard is *required* on `StallF`; without it the PC freezes and the core re-fetches the wrong-path instruction indefinitely. On `StallD` the guard is *redundant*: `FlushD=PCSrcE` already clears the F/D register, and `pipe_reg` prioritises flush over stall (`rst/flush > stall > load`), so the register bubbles regardless of `StallD`; it is kept only for symmetry. In forwarding mode `stallD_i` (a load in E) and `PCSrcE` (a taken branch/jump in E) are mutually exclusive, so the guard only ever fires in stall-only mode, where a `JAL`/`JALR` both writes `rd` and sets `PCSrcE`.

4 ISA coverage and control signals

4.1 Instruction groups

The core targets a practical RV32I subset intended to be "GCC-friendly" for freestanding bare-metal programs:

- Control flow: JAL, JALR, and all RV32I branches (BEQ/BNE/BLT/BGE/BLTU/BGEU).
- Address materialization: LUI, AUIPC.
- Loads: LB/LBU, LH/LHU, LW (with sign/zero extension).
- Stores: SB/SH/SW (with correct byte-lane strobes and alignment).
- Integer ALU: OP-IMM and OP arithmetic/logical instructions (ADD/SUB/AND/OR/XOR/SLT/SLTU/SLL/SRL/SRA and immediate forms).

4.2 Control signal glossary (selected)

The controller emits decode-stage signals that are registered into E/M/W. Common signals you will see in waveforms and RTL include:

- `ResultSrcW`: writeback mux select (ALU vs MEM vs PC+4).
- `MemWriteM` / `MemReadM`: DMEM write/read enables.
- `ImmSrcD`: immediate format (I/S/B/U/J).
- `ALUControlE`, `SrcASelE`, `ALUSrcBE`: ALU operation and operand selection.
- `BranchOpE`: branch comparator mode (eq/lt/ltu/etc.).
- `JumpE` / `JalrE`: unconditional control-flow redirect (JAL/JALR).
- `PCSrcE`: redirect decision (taken branch/jump) that selects the next PC.

4.3 Control naming and stage suffixes

Control is generated in Decode and registered through E/M/W (for example `ImmSrcD`, `ALUControlE`, `MemWriteM`, `ResultSrcW`). This mirrors the textbook convention while making timing explicit in the RTL.

4.4 Compact control matrix (from `docs/instruction_control.md`)

Execute + writeback

Group	ImmSrcD	SrcASelE	ALUSrcBE	ALUControlE	RegWriteW	ResultSrcW
LUI	IMM_U	OPA_ZERO	imm	ALU_ADD	yes	WB_ALU
AUIPC	IMM_U	OPA_PC	imm	ALU_ADD	yes	WB_ALU
JAL	IMM_J	OPA_PC	imm	ALU_ADD	yes	WB_PC4
JALR	IMM_I	OPA_RS1	imm	ALU_ADD	yes	WB_PC4
Branch	IMM_B	OPA_RS1	rs2	ALU_SUB	no	WB_ALU
Load	IMM_I	OPA_RS1	imm	ALU_ADD	yes	WB_MEM
Store	IMM_S	OPA_RS1	imm	ALU_ADD	no	WB_ALU
OP-IMM	IMM_I	OPA_RS1	imm	funct	yes	WB_ALU
OP	IMM_I	OPA_RS1	rs2	funct	yes	WB_ALU

Memory + PC control

Group	MemReadM	MemWriteM	MemSizeM	MemUnsignedM	BranchOpE	JumpE	JalrE
LUI	0	0	–	–	BR_NONE	0	0
AUIPC	0	0	–	–	BR_NONE	0	0
JAL	0	0	–	–	BR_NONE	1	0
JALR	0	0	–	–	BR_NONE	1	1
Branch	0	0	–	–	funct	0	0
Load	1	0	size	funct	BR_NONE	0	0
Store	0	1	size	0	BR_NONE	0	0
OP-IMM	0	0	–	–	BR_NONE	0	0
OP	0	0	–	–	BR_NONE	0	0

5 Memory map, linker script, and GCC-to-hex flow

5.1 Harvard memory map

The RTL models separate instruction and data memories. Software is linked to match the address split so code and writable data never overlap.

Region	Base address	Size	Access	Purpose
IMEM	`0x0000_0000`	8 KiB	read/execute	`.text`, `.init`, `.rodata`
DMEM	`0x2000_0000`	8 KiB	read/write	`.data`, `.sdata`, `.bss`, stack, signature window

Tests use a fixed signature scratch region at `0x2000_0100`: programs write deterministic 32-bit words there, and the testbench compares them against golden `.sig` files.

5.2 Why these addresses?

The documentation emphasizes a few motivations:

- The PC resets to zero, so placing IMEM at `0x0000_0000` makes reset-to-first-instruction trivial.
- The DMEM base at `0x2000_0000` makes the Harvard split obvious from the address, and leaves room to grow toward a small SoC/MMIO map.
- The linker fixes a deterministic stack top and a deterministic signature window for self-checking tests.

5.3 A note on address decoding in the simple memories

The VHDL `imem_dp` and `dmem` models intentionally do not implement full high-bit address decode; they mask down to the configured depth when indexing their arrays. The software *still* uses full logical addresses (IMEM at `0x0000_xxxx`, DMEM at `0x2000_xxxx`) so that the linker keeps sections separated. When integrating into a larger system, you can wrap proper decode logic around the memories without changing the compiler/linker view.

5.4 Data-side reads from the text region (`.rodata`)

Because `.rodata` constants are placed in IMEM alongside `.text`, loads that target addresses below `DMEM_BASE_ADDR` (`0x2000_0000`) must read from IMEM, not DMEM. The top-level wrapper `top.vhd` instantiates a true dual-port instruction memory (`imem_dp`): port A serves instruction fetch and port B serves data-side reads. A registered mux-select (`use_imem_w`) aligns the IMEM/DMEM read-data mux with the synchronous BRAM output timing.

5.5 Linker script (`link.ld`)

The provided linker script enforces the memory map:

- `.text` and `.rodata` go to IMEM at `0x0000_0000` (the PC resets to zero).
- `.data` (and small-data `.sdata`) live in DMEM at runtime but are placed *in the ELF load image* so the build can initialize DMEM before releasing reset.
- `.bss` (and `.sbss`) are zero-initialized at runtime and do not need hex initialization data.

- `__stack_top` is set to the top of DMEM so freestanding startup code can initialize `sp`.
- `.note` sections are discarded, and the link stops with an error if `.text` does not start at `0x0000_0000`. Some toolchains, for example the RISC-V GCC that comes with AMD Vitis, add a build-ID note section by default; placed in front of `.text`, it would move the code away from the address where the IMEM image starts.

Key snippet

```
MEMORY {
  IMEM (rx) : ORIGIN = 0x00000000, LENGTH = 8K
  DMEM (rw) : ORIGIN = 0x20000000, LENGTH = 8K
}
```

Spike variant (`link_spike.ld`) The golden signatures are generated on the Spike ISA simulator (Section 6.9). Spike reserves the low addresses for its debug module and boot ROM, so `link_spike.ld` places the code at `0x8000_0000` and keeps DMEM, the stack, and the signature window at their MinRV32 addresses. `.data` is placed at its run address, so Spike's ELF loader preloads DMEM as `<name>.data.hex` does on the core. `build_prog.sh` selects it through the `LINKER_SCRIPT` environment variable. Section 6.9 explains which signature values this affects.

5.6 GCC → ELF → hex images

The project's workflow emphasizes real software running on the RTL:

1. Compile/link a freestanding RV32I program with a bare-metal RISC-V GCC toolchain.
2. Extract `.text/.rodata` and `.data/.sdata` into raw binaries via `objcopy`.
3. Convert those binaries into newline-delimited 32-bit hex words (`XXXXXXXX` per line) for IMEM/DMEM initialization.

The helper script `scripts/build_prog.sh` automates this and produces: `<base>.elf`, `<base>.text.hex`, `<base>.data.hex` (and, for C sources, a compiler-generated assembly listing `<base>.S`).

Practical guard: do the images fit? The educational memories are small (default 8 KiB IMEM and 8 KiB DMEM). The build script prints a size summary for the generated `.text` and `.data` images and emits warnings if either exceeds the configured IMEM/DMEM capacity. If you change the VHDL memory sizes, set `IMEM_BYTES` and `DMEM_BYTES` in the environment so the warnings match your configuration.

5.7 Future work: one memory map for the core and Spike

The two linker scripts exist only because the core's IMEM sits at `0x0`, where Spike has its debug module. Spike does not accept memory there. Its other devices (boot ROM at `0x1000`, CLINT at `0x0200_0000`, PLIC at `0x0C00_0000`, UART at `0x1000_0000`) do not overlap DMEM at `0x2000_0000`. If IMEM and the reset PC move to `0x8000_0000` (the RAM base also used by riscv-tests, riscv-arch-test, QEMU `virt`, and Sail), one linker script serves both. DMEM, the stack, and the signature window do not change. The current layout does not affect the golden signatures as long as tests follow the code-address rule (Section 6.9), so this change is not needed now. Required changes:

1. **Reset PC.** Add a reset-value generic to `pipe_reg` (default zero) and a `RESET_PC` generic (default `x"80000000"`) passed down to `pc_reg` in `fetch_stage`.

2. **Load-path decode.** `top.vhd` and `sim/top_trace.vhd` send a load to IMEM port B when `addr < DMEM_BASE_ADDR`; test address bit 31 instead (set for IMEM, clear for DMEM), which also needs less logic.

3. **Memories.** No change: `imem_dp` and `dmem` index only the low address bits.

4. **Linker script.** Set `IMEM ORIGIN = 0x80000000` in `link.ld` and remove `AT>IMEM` (nothing copies `.data` from its load address). Delete `link_spike.ld` and the `LINKER_SCRIPT` override in `gen_golden.sh`.

5. **Documentation.** Update the address tables, remove the code-address rule, and update examples that show PC values.

Raw code addresses would then give the same signature on both, and the existing `.sig` files stay valid. The PC would no longer reset to zero. The differences in Section 6.9 remain.

6 Directed instruction tests and helper scripts

6.1 ISA tests

Directed tests under `tests/isa/` exercise every supported instruction group. Each test writes a deterministic signature to `0x2000_0100` and ships with a golden `.sig` file. The golden words are generated with the Spike RISC-V ISA simulator (Section 6.9).

6.2 Prerequisites

The documented workflow assumes:

- A bare-metal RISC-V GCC toolchain (e.g. `riscv64-unknown-elf-gcc` and `objcopy`).
- Python 3 (used for binary-to-hex conversion).
- GHDL with VHDL-2008 support (for simulation).

On Ubuntu, the documentation suggests installing the basics via APT:

```
sudo apt update
sudo apt install gcc-riscv64-unknown-elf binutils-riscv64-unknown-elf \
  ghdl python3 python3-pip make
```

Optional: Spike (golden signatures) `gen_golden.sh` and `check_goldens.sh` need the Spike ISA simulator. Spike is not packaged for Ubuntu, so it is built from source:

```
sudo apt install -y build-essential git device-tree-compiler \
  libboost-regex-dev libboost-system-dev
git clone https://github.com/riscv-software-src/riscv-isa-sim.git
cd riscv-isa-sim && mkdir build && cd build
../configure --prefix=/opt/riscv   # or --prefix=$HOME/.local (no sudo)
make -j"$(nproc)" && sudo make install
export PATH=/opt/riscv/bin:$PATH
spike --help | grep debug-cmd      # check
```

Spike is needed only to create or re-check `.sig` files, not to run the existing tests.

Optional: schematic generation and DigitalJS interactive simulation To generate SVG schematics of RTL entities and to export DigitalJS JSON for interactive browser-based simulation (see `scripts/gen_schematic.sh`):

```
sudo apt install -y yosys ghdl yosys-plugin-ghdl graphviz
sudo apt install -y nodejs npm
sudo npm install -g netlistsvg
sudo npm install -g yosys2digitaljs
```

6.3 One-test and batch workflows

The repository provides scripts that build programs (using GCC) and run the self-checking testbench (GHDL):

```
./scripts/run_test.sh full_core
./scripts/run_test.sh --trace full_core
./scripts/run_test.sh --fst full_core
./scripts/run_tests.sh
./scripts/run_tests.sh --trace
```

Useful environment overrides include `MAX_CYCLES` (for longer programs), `FORWARDING_ENABLE=false` (stall-only hazards), and tool-path variables such as `RISCV_GCC` and `RISCV_OBJCOPY`. For simulation, the scripts also honor `GHDL` and `GHDL_STD` and provide a `-verbose`/`VERBOSE=1` mode to echo the underlying GHDL commands. To keep the repository root clean (and to avoid collisions when running multiple tests in parallel), the scripts isolate GHDL work libraries and elaboration executables under `build/ghdl/` by default (override with `GHDL_WORKDIR`).

6.4 Waveform dumps (GTKWave)

For waveform-first debugging, the scripts can dump waveforms directly from GHDL:

- `-fst`: fast, compact waveform format (recommended).
- `-vcd`: widely supported text waveform.
- `-wave`: GHDL's `.ghw` format.

```
./scripts/run_test.sh --fst addi_only
./scripts/view_wave.sh build/waves/addi_only.fst
```

6.5 Unit-level testbenches

For component bring-up without building a program image, run unit testbenches:

```
./scripts/run_unit_tb.sh regfile
./scripts/run_unit_tb.sh --fst alu
```

6.6 DigitalJS interactive simulation

`gen_schematic.sh -digitaljs` exports a DigitalJS-compatible JSON that you can upload to https://digitaljs.tilk.eu/. DigitalJS is both a schematic viewer *and* an interactive simulator: you drive inputs by clicking and watch values propagate through the gates. It acts as a visual complement to GTKWave (for leaf modules) and to the GHDL full-core tests (for the whole pipeline). It is not a replacement for the signature-checked `run_test.sh` / `run_tests.sh` flows – those remain the authoritative pass/fail.

Automatic clock and reset ergonomics The `-digitaljs` post-processor calls `yosys2digitaljs`'s `io_ui()` helper, which remaps raw port devices into simulator-UI widgets so the exported circuit is immediately runnable without manual clock toggling: a 1-bit `Input` named `clk` becomes a self-ticking `Clock` device, any other 1-bit `Input` (e.g. `reset`) becomes a one-click `Button`, multi-bit `Inputs` become `NumEntry` typeable fields, and outputs become `Lamp` / `NumDisplay`. Upload, press the simulator's run button (auto-pressed by default when loading the .json file), and click `reset` once and then click again to create a reset pulse – the clock runs on its own.

Simple modules (leaf entities) Export a single entity and open it in DigitalJS:

```
./scripts/gen_schematic.sh --digitaljs alu
./scripts/gen_schematic.sh --digitaljs regfile
./scripts/gen_schematic.sh --digitaljs extend
```

Upload `build/schematics/<entity>.digitaljs.json` and toggle inputs.

Full pipeline with real firmware (`-mem-init`) Yosys synthesis excludes the `synthesis translate_off` guards in `mem_init_pkg.vhd`, so every memory cell in the exported netlist has an all-zero initial state. The `-mem-init` flag patches the `$mem`/`$mem_v2` INIT parameter with the real hex images before `yosys2digitaljs` converts the netlist, giving the DigitalJS interactive simulation a faithful firmware image:

```
./scripts/build_prog.sh tests/isa/full_core.c full_core
./scripts/gen_schematic.sh --digitaljs \
    --mem-init imem_dp=build/full_core.text.hex \
    --mem-init dmem=build/full_core.data.hex \
    top
```

Upload the resulting `build/schematics/top.digitaljs.json` to https://digitaljs.tilk.eu/ to step through the pipeline with the program already loaded in memory (Figure 10). The `-mem-init` injection only works on the hierarchical (default) netlist where `imem_dp` and `dmem` are distinct modules; under `-flat` the modules merge into the top level and the injection becomes a no-op (a warning is printed).

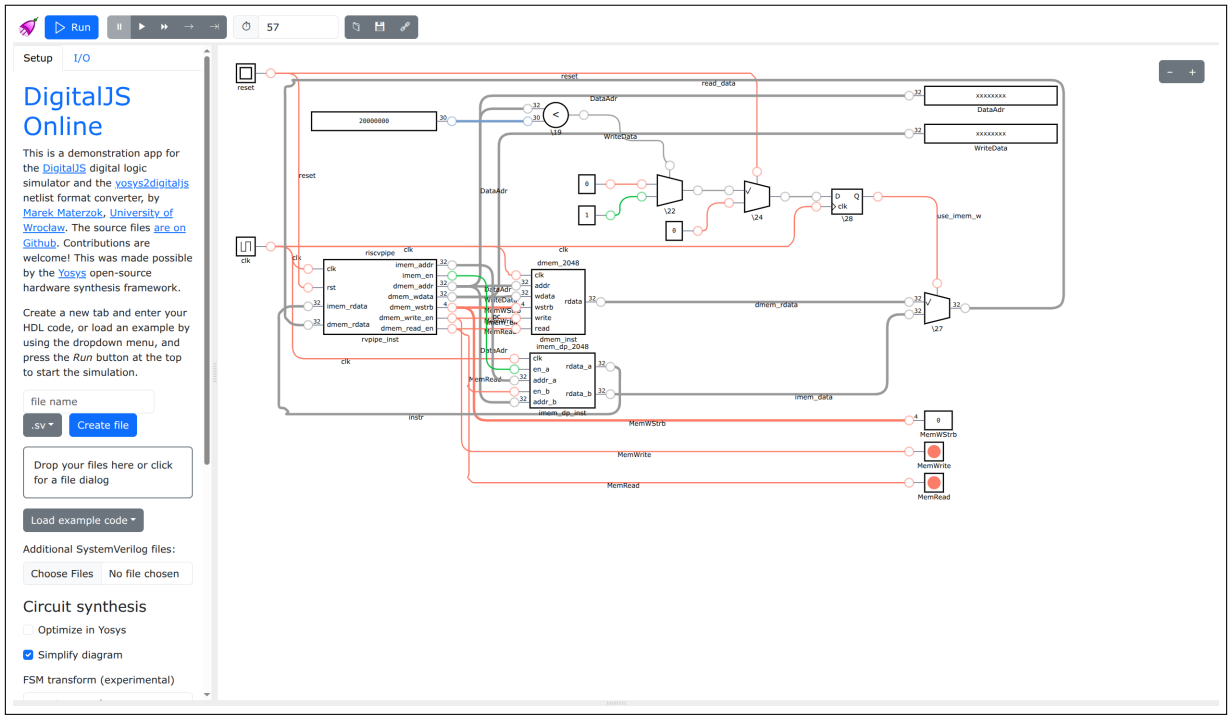


Figure 10: DigitalJS simulation of the full_core.c program running on MinRV32.

6.7 Trace output (optional)

Both the GHDL-driven scripts and the Vivado documentation describe an optional per-instruction trace that prints decoded assembly and key control selections grouped by stage. This is designed as a teaching aid: it links what the program is doing (assembly) to what the controller and hazard logic are doing (mux and enable choices).

When the hazard unit takes action (stall/flush/forward), the trace prints an additional `HAZARD:` line showing stall/flush decisions plus the forwarding-source selects for the E-stage operands:

```
HAZARD: StallF=stall StallD=stall FlushD=keep FlushE=flush FwdA=rf FwdB=rf
```

`FwdA/FwdB` select `rf` (regfile), `M` (Memory stage), or `W` (Writeback stage). The hazard line is only printed when something is asserted or forwarded.

6.8 Authoring new tests

New tests can be written as assembly (`.S`) or freestanding C (`.c`) under `tests/isa/`. The expected flow is:

1. Write a deterministic workload using only the supported instruction subset.
2. Write final observation words into the signature buffer at `0x2000_0100`.
3. Generate the golden signature with Spike: `./scripts/gen_golden.sh -words N -write <name>` runs the same program on the Spike reference model, reads `N` words from the signature window at `0x2000_0100`, and writes `tests/isa/<name>.sig` (Section 6.9). Do not store raw code or `.rodata` addresses in the signature (Section 6.9). (Compiling the C for the host x86 architecture is *not* a reliable oracle, since 32-bit overflow and shift semantics may differ from RV32I.)
4. Re-run via `scripts/run_test.sh <name>` for regression.

6.9 Golden signatures with Spike

The expected `.sig` words come from Spike, the RISC-V reference ISA simulator. Spike does not depend on the RTL, so it is an independent oracle for the design under test.

```
./scripts/gen_golden.sh factorial                 # print, save build/golden/factorial.
    sig
./scripts/gen_golden.sh --check factorial         # compare with tests/isa/factorial.sig
./scripts/gen_golden.sh --words 4 --write my_test # create tests/isa/my_test.sig
./scripts/check_goldens.sh                        # check every tests/isa/*.sig
```

`gen_golden.sh` builds the test with `build_prog.sh` (same compiler flags as the core) and the linker script `link_spike.ld`. It runs the ELF on Spike in debug mode for `SPIKE_MAX_INSNS` instructions (default 1 000 000) and checks that the PC is parked in the final self-loop. It then reads the signature window and writes `build/golden/<name>.sig`, one lowercase hex word per line. The word count is `-words N` or the line count of the existing `.sig` file. `-write` copies the result to `tests/isa/<name>.sig`, and `-check` compares it with that file. `check_goldens.sh` runs `-check` for every test. The environment variables `SPIKE`, `SPIKE_ISA` (default `rv32i`), and `SPIKE_MAX_INSNS` select the binary, the ISA string, and the instruction budget.

Spike vs MinRV32 code addresses Spike reserves the low address space for its debug module (`0x0000_0000`–`0x0000_0FFF`) and boot ROM (from `0x1000`), so instruction memory cannot sit at `0x0` as it does on MinRV32. `link_spike.ld` moves the code to `0x8000_0000`; data memory does not move.

Item	MinRV32 (`link.ld`)	Spike (`link_spike.ld`)
`.init`, `.text`, `.rodata`	`0x0000_0000`	`0x8000_0000`
`.data`, `.bss`, stack top	`0x2000_0000`… / `0x2000_2000`	same
signature window	`0x2000_0100`	same

Stores to the signature window go to the same address on both, so only the stored *value* can differ. A golden word is affected when the value is, or is computed from, the address of code or `.rodata`; under Spike it is larger by `0x8000_0000`. Affected values include:

- function pointers (`(uint32_t)&main`);
- label addresses (`la t0, label` followed by a store of `t0`);
- addresses of string literals or `const` tables;
- raw `jal`/`jalr` link values and raw `auipc` results.

Unaffected values are arithmetic and logic results, values loaded *from* `.rodata` (as in `lui_auipc.S`), data-memory addresses, and differences between two code addresses (as in `branch_jump.S`). To use a code address in a test, store an offset such as `(uint32_t)&f - (uint32_t)&_start`, which is the same under both links. A program that stores `(uint32_t)_start + 4` and `(uint32_t)&main - (uint32_t)_start` gives `00000004 00000010` on the core and `80000004 00000010` on Spike. A `-check` mismatch of exactly `0x8000_0000` means that a code address has leaked into the signature. Moving the core's code to `0x8000_0000` would remove this difference (Section 5.7).

Other Spike vs MinRV32 differences In the cases below, the two simulators run the same program differently, so a golden word can differ even though both run the same code. None of the current tests hits them.

Register values at `_start`. Spike starts in its boot ROM, which leaves `t0` = `0x8000_0000` (the entry address), `a1` = `0x0000_1020` (the device-tree address), and `a0` = `0` (`mhartid`). On the core every register is `0` after reset. Write every register before you read it; do not rely on reset values.

Cases the core does not trap but Spike does. The core has no exceptions. Spike raises one in the cases below. Because `mtvec` is `0`, Spike then stays at PC `0x0000_0000`.

Case	MinRV32	Spike
Misaligned load/store	Ignores the low address bits (a word uses the aligned word, a halfword uses address bit 1): `lw` from `0x2000_0109` reads `0x2000_0108`.	Misaligned exception
Outside the memory windows	Wraps the address into the 8 KiB array (`0x2000_2000` reads `0x2000_0000`). Loads below `0x2000_0000` read IMEM; stores always write DMEM.	Access fault
`ecall`, `ebreak`, undecoded	No-op	Exception

CSR instructions do not assemble with `-march=rv32i`, so they cannot reach either simulator. `gen_golden.sh` does not detect a trap: the trap loop at PC `0x0` looks like the final self-loop, so the script reads whatever the signature window holds at that point, often zeros. In `build/golden/<name>.spike.log` the two PC values are an address in `0x8000_xxxx` after a normal run and `0x00000000` after a trap.

6.10 Running a program on Spike by hand

Spike does not compile anything. It is an instruction-set simulator: it loads an ELF file of RISC-V machine code and executes it one instruction at a time. A C or assembly source is first compiled and linked by the RISC-V GCC toolchain, and the resulting ELF is then run on Spike. `gen_golden.sh` does these steps automatically; by hand, from the repository root:

```
# 1. Compile and link for Spike (code at 0x8000_0000); .c or .S
LINKER_SCRIPT=$PWD/link_spike.ld ./scripts/build_prog.sh \
    tests/isa/factorial.c factorial.spike
# 2. Machine code and symbol addresses
riscv64-unknown-elf-objdump -d build/factorial.spike.elf | less
riscv64-unknown-elf-nm build/factorial.spike.elf   # 80000068 T main
# 3. Start Spike in its interactive debugger
spike -d --isa=rv32i -m0x80000000:0x2000,0x20000000:0x2000 \
    build/factorial.spike.elf
```

`-d` starts the interactive debugger, halted before the first instruction. `-isa=rv32i` selects the base ISA of the core. `-m...` creates two 8 KiB memories, code at `0x8000_0000` and data at `0x2000_0000` (the MinRV32 DMEM, so the signature window is at `0x2000_0100`). The warning `tohost and fromhost symbols not in ELF` is expected: the tests are freestanding, have no `printf`, and report their results in the signature window. The tests never exit (they end in a self-loop), so without `-d` Spike runs until Ctrl+C. The same commands can be run in batch with `-debug-cmd=<file>`, which is how `gen_golden.sh` works (its file is `build/golden/<name>.spike.cmd`).

6.11 Stepping through a program in Spike's debugger

Spike shows a `:` prompt. Execution begins in the boot ROM at `0x1000`, which runs five instructions and jumps to `_start` at `0x8000_0000`. Most commands take a core number, always `0` here; `help` lists all of them.

Command	Effect
Enter	run one instruction and print it (`run 1`)
`run N` / `r N`	run `N` instructions, printing each one
`rs N`	run `N` instructions silently
`run`	run until Ctrl+C, which returns to the prompt
`pc 0`, `insn 0`	show the PC, or the instruction at the PC
`reg 0`, `reg 0 a0`	show all registers, or one (ABI names work)
`mem 0x20000100`	show memory at a physical address (signature word 0)
`until pc 0 0x80000068`	run until the PC reaches an address (here `main`)
`untiln pc 0 <addr>`	the same, printing each instruction
`until reg 0 a0 0x2d0`	run until a register holds a value
`until mem 0x20000100 0x2d0`	run until a memory word holds a value
`q`	quit

A short session with `factorial.c` (Spike output after each command):

```
: pc 0
0x00001000                              # in the boot ROM
: r 3
core   0: 0x00001000 (0x00000297) auipc   t0, 0x0
core   0: 0x00001004 (0x02028593) addi    a1, t0, 32
core   0: 0x00001008 (0xf1402573) csrr    a0, mhartid
: until pc 0 0x80000068                 # run to main
: insn 0
0xff010113 addi    sp, sp, -16
: reg 0 sp
0x20002000                              # stack top (__stack_top)
: reg 0 ra
0x8000000c                              # return address: self-loop in _start
: r 2
core   0: 0x80000068 (0xff010113) addi    sp, sp, -16
core   0: 0x8000006c (0x00600513) li      a0, 6
: until mem 0x20000100 0x2d0            # until sig[0] = 6! is stored
: pc 0
0x80000080                              # the instruction after the store
: mem 0x20000100
0x000002d0                              # the value in factorial.sig
: q
```

The PCs printed by Spike are `0x8000_0000` higher than on the core because of the Spike link (Section 6.9); the instructions and the data values are the same. `./scripts/run_test.sh -trace factorial` shows the same instructions on the core as they pass through the five pipeline stages.

6.12 Script roles (high level)

- `build_prog.sh`: compile and emit `.text.hex`/`.data.hex` under `build/` (and print a size summary for IMEM/DMEM).
- `run_test.sh`: build one test, run the VHDL testbench, compare signatures, stop on first mismatch.
- `gen_golden.sh`: run one test on Spike and produce its golden signature (`-write` to update `tests/isa/<name>.sig`, `-check` to compare with it).
- `check_goldens.sh`: run `gen_golden.sh -check` for every test with a `.sig` file (match/differ summary).
- `run_tests.sh`: iterate over all tests with `.sig` files for regression runs (optional `-trace` / `-verbose`).
- `clean.sh`: remove build and simulator artifacts (`build/`, `tmp/` if present, and any `work-obj*.cf`/`e~*` leftovers).
- `bin_to_hex.py`: convert raw little-endian binaries into newline-delimited 32-bit hex words (advanced: `-word-bytes`).
- `run_unit_tb.sh`: build and run a single unit-level testbench from `sim/unit/` (e.g. `alu`, `regfile`). Accepts `-fst`/`-vcd`/`-wave` for waveform dumps.

- `run_unit_tests.sh`: batch driver that iterates over every `sim/unit/tb_*.vhd` testbench and calls `run_unit_tb.sh` for each one (pass/fail summary at the end).
- `view_wave.sh`: convenience wrapper that opens a waveform file in GTKWave (if installed).
- `gen_schematic.sh`: generate SVG schematics of RTL modules via Yosys (GHDL plugin) + netlistsvg. Supports depth-limited flattening (`-flat N`), selective sub-module expansion (`-expand <entity>`), and memory initialisation for DigitalJS simulation (`-mem-init <module>=<hex>`). Requires extra packages; see above.
- `mem_init_inject.py`: patch a Yosys JSON netlist by writing hex-file contents into the `INIT` parameter of `$mem`/`$mem_v2` cells. Called automatically by `gen_schematic.sh -mem-init`; can also be used standalone.
- `vhdl_sources.sh`: shared helper sourced by the simulation scripts to provide ordered VHDL compile lists (`VHDL_RTL_SOURCES`, `VHDL_SIM_SOURCES`). Not executed directly.

7 Vivado/XSim and FPGA bring-up

Vivado is a common next step after GHDL validation: it can simulate with XSim, then synthesize/implement the same RTL on a Xilinx FPGA. XSim simulation plays the same role here as the GHDL flow used earlier in the project (compile RTL plus testbench, run a behavioural simulation, check the signature) – the two flows are interchangeable verification steps that consume the same RTL and the same `tests/isa/<name>.sig` golden files. For pre-Vivado interactive exploration of the pipeline in a browser (schematic view plus drag-and-drop simulation, with real firmware loaded via `-mem-init`), see Section 6.6 (DigitalJS interactive simulation). DigitalJS is a useful visual companion before committing to a Vivado project setup, but it is not signature-checked and does not replace XSim as a verification step.

Automated project setup The full Vivado bring-up flow – project creation, source import, VHDL-2008 file-type override, top selection, generics, and board XDC attachment – is captured by a single Tcl driver, `scripts/vivado_project.tcl`, and documented end-to-end in `docs/vivado_instructions.md` (XSim simulation, synthesis/implementation, and on-board ILA validation). The sections below summarise what that script does and show the relevant Tcl snippets so the manual steps remain reproducible if you prefer GUI clicks over Tcl.

```
# Open project in the GUI, ready to launch sim/synth:
vivado -source scripts/vivado_project.tcl -tclargs full_core basys3
# Non-interactive (batch) creation:
vivado -mode batch -source scripts/vivado_project.tcl -tclargs full_core zedboard
```

7.1 Vivado simulation (XSim) checklist

The Tcl driver `scripts/vivado_project.tcl` performs every step of this checklist; the manual list below is mainly useful for understanding what the script does (and what to tweak in the GUI if you ever set things up by hand).

1. Build the program hex images (example target: `tests/isa/full_core.c`; expected signature: `tests/isa/full_core.sig`):

   ```
   ./scripts/build_prog.sh tests/isa/full_core.c full_core
   ```

2. Create the project and import sources via the Tcl driver:

   ```
   vivado -source scripts/vivado_project.tcl -tclargs full_core basys3
   ```

   Internally this adds every `rtl/*.vhd` as a design source and every `sim/*.vhd` as a simulation source (corresponding Tcl excerpts shown below in Section 7.2).

3. Force VHDL-2008 on the imported sources (Vivado defaults to VHDL-93). The driver does this automatically (see the snippet in Section 7.3).

4. Set the simulation top and point the testbench generics at the generated hex files and the expected signature file (the driver does this too; see Section 7.4).

5. Run behavioral simulation (`launch_simulation`) and look for `Note: TEST PASSED`. The driver sets the simulation run time to `all`, so the simulation runs until the testbench stops it (Vivado's default of 1000 ns ends before `full_core` passes at 28335 ns).

7.2 Vivado source organization tip (exact lists)

The documentation includes two exact source lists: one for synthesis (design top `rtl/fpga_top.vhd` – a thin wrapper around `rtl/top.vhd` that exposes only `clk` / `reset` on the pad ring), one for simulation (simulation top `sim/tb_minrv32.vhd`, which drives `top` directly so the testbench still sees the observation ports).

In practice you do not type these lists into the GUI; the Tcl driver imports them with two `glob` calls – `rtl/*.vhd` into the design fileset, `sim/*.vhd` into `sim_1` – and then selects the correct top for each fileset:

```
# scripts/vivado_project.tcl (excerpt)
set rtl_files [glob -nocomplain [file join $repo rtl *.vhd]]
add_files -norecurse $rtl_files
set sim_files [glob -nocomplain [file join $repo sim *.vhd]]
add_files -fileset sim_1 -norecurse $sim_files

set_property top fpga_top   [current_fileset]
set_property top tb_minrv32 [get_filesets sim_1]
```

The exact file lists below are useful as a manual reference: anything pulled in by the `glob` above appears here.

Synthesis sources

```
rtl/alu.vhd
rtl/branch_unit.vhd
rtl/controller_pipe.vhd
rtl/core_pkg.vhd
rtl/mem_init_pkg.vhd
rtl/decode_stage.vhd
rtl/dmem.vhd
rtl/execute_stage.vhd
rtl/extend.vhd
rtl/fetch_stage.vhd
rtl/hazard_unit.vhd
rtl/imem_dp.vhd
rtl/load_extend_unit.vhd
rtl/maindec.vhd
rtl/aludec.vhd
rtl/mem_access_unit.vhd
rtl/memory_stage.vhd
rtl/mux2.vhd
rtl/mux3.vhd
rtl/pipe_reg.vhd
rtl/regfile.vhd
rtl/riscvpipe.vhd
rtl/datapath_pipe.vhd
rtl/top.vhd
rtl/fpga_top.vhd
rtl/writeback_stage.vhd
```

Note `top.vhd` instantiates `imem_dp` (dual-port BRAM) for instruction memory and for data-side reads from the text region (`.rodata` loads). Port B can be left disabled if you want a logically

single-port instruction memory. `fpga_top.vhd` is the Vivado synthesis top: it instantiates `top` and XOR-reduces the five observation output ports into a single 1-bit `led` so the FPGA pad ring only sees `clk` / `reset` / `led` (3 IOs instead of 72). The XOR fold is deliberate: it forces every observation bit to reach a pad, so Vivado cannot prune the datapath as dead code during backwards optimisation. The fold is XORed with the MSB of a free-running 24-bit counter (about 4.8 Hz at 80 MHz), so the LED blinks visibly whenever the core clock is running.

Core clock The core does not run at the board clock. `fpga_top.vhd` contains an MMCM (Mixed-Mode Clock Manager, the clock generator block inside 7-series FPGAs) that makes the core clock: core clock = board clock × `MMCM_MULT` / (`MMCM_DIVCLK` × `MMCM_OUT_DIV`). The Tcl driver computes these generics from the core clock you ask for with `-clk-mhz` (default 80 MHz on Basys3 and ZedBoard) and picks the highest frequency that is not above it. The board XDC constrains only the 100 MHz board clock; Vivado derives the core clock constraint from it, so the timing report checks the real core clock. The core is held in reset until the MMCM output is stable. At 80 MHz the worst setup slack is only about 0.17 ns on both boards (Section 7.11); if a change makes it negative, use 75 MHz, as below.

```
vivado -mode batch -source scripts/vivado_project.tcl \
       -tclargs full_core basys3 --clk-mhz 75
```

Simulation sources (with optional trace wrappers)

```
rtl/alu.vhd
rtl/branch_unit.vhd
rtl/controller_pipe.vhd
rtl/core_pkg.vhd
rtl/mem_init_pkg.vhd
rtl/decode_stage.vhd
rtl/dmem.vhd
rtl/execute_stage.vhd
rtl/extend.vhd
rtl/fetch_stage.vhd
rtl/hazard_unit.vhd
rtl/imem_dp.vhd
rtl/load_extend_unit.vhd
rtl/maindec.vhd
rtl/aludec.vhd
rtl/mem_access_unit.vhd
rtl/memory_stage.vhd
rtl/mux2.vhd
rtl/mux3.vhd
rtl/pipe_reg.vhd
rtl/regfile.vhd
rtl/riscvpipe.vhd
rtl/datapath_pipe.vhd
sim/riscvpipe_trace.vhd
rtl/top.vhd
sim/top_trace.vhd
sim/tb_minrv32.vhd
rtl/writeback_stage.vhd
```

7.3 VHDL-2008 enable (Tcl snippet)

```
set repo <repo_root>
foreach f [glob -nocomplain $repo/rtl/*.vhd] {
  set_property file_type {VHDL 2008} [get_files $f]
}
foreach f [glob -nocomplain $repo/sim/*.vhd] {
  set_property file_type {VHDL 2008} [get_files $f]
}
```

7.4 Pointing the testbench at hex/signature files

Vivado can set testbench generics on the `sim_1` fileset. The documented example configures IMEM/DMEM init files plus an expected signature file:

```
set_property top tb_minrv32 [get_filesets sim_1]
set repo      <repo_root>
set build_dir [file join $repo build]
set_property generic \
  "IMEM_INIT_FILE=\"$build_dir/full_core.text.hex\" \
   DMEM_INIT_FILE=\"$build_dir/full_core.data.hex\" \
   EXPECTED_FILE=\"$repo/tests/isa/full_core.sig\" \
   SIGNATURE_WORDS=10 \
   MAX_CYCLES=5000 \
   HALT_ON_PASS=true" \
  [get_filesets sim_1]
```

`SIGNATURE_WORDS` must match the number of 32-bit words in the `.sig` file (one per line). For example, `full_core.sig` uses 10 words. The Tcl driver `scripts/vivado_project.tcl` computes this automatically by counting non-blank lines in the `.sig` file.

7.5 Optional: per-instruction trace wrappers

For a more didactic simulation transcript, the project includes simulation-only wrappers that print decoded assembly and key control choices. In Vivado, add `sim/top_trace.vhd` and `sim/riscvpipe_trace.vhd` to simulation sources; for synthesis, omit them so the synthesizable RTL remains unchanged.

When the hazard unit takes action (stall/flush/forward), the trace also prints a `HAZARD:` line with the stall/flush decisions and the forwarding selects:

```
HAZARD: StallF=stall StallD=stall FlushD=keep FlushE=flush FwdA=rf FwdB=rf
```

`FwdA/FwdB` select `rf`, `M`, or `W`.

7.6 Expected warnings

Because the simple memories mask addresses down to their depth, Vivado may warn that higher address bits are unused. Similar "unused" warnings can appear for helpers that only consume low address bits for byte strobes and alignment. The documentation calls these warnings expected for this style of educational memory model.

7.7 BRAM initialization: simulation vs synthesis

The memory models share a single init mechanism: an impure `init_mem()` function in `rtl/mem_init_pkg.vhd` reads a plain-text hex file at *elaboration time* and returns a pre-populated `word_mem_t` array.

- Simulation (GHDL/XSim): the function runs in the simulator's elaboration phase and pre-loads the array before time 0.
- Synthesis (Vivado): the same code path executes in Vivado's elaborator – TEXTIO is supported at elaboration – and the resulting array is baked into the inferred BRAM's `INIT_xx` parameters in the bitstream. No tool-specific `INIT_FILE` attribute is needed.

Provide absolute paths to the hex files via the top-level `IMEM_INIT_FILE` / `DMEM_INIT_FILE` generics (relative paths fail because the elaborator runs from Vivado's project working directory, not the repo root).

7.8 Synthesis top generics (init files and sizes)

The synthesis top is `fpga_top`. Its generics are `IMEM_WORDS`, `DMEM_WORDS`, `IMEM_INIT_FILE`, `DMEM_INIT_FILE`, `FORWARDING_ENABLE`, and the clock settings `CLK_IN_PERIOD_PS`, `MMCM_MULT`, `MMCM_DIVCLK`, and `MMCM_OUT_DIV` (the `DMEM_BASE_ADDR` split is fixed inside `rtl/top.vhd` at `0x2000_0000` and is not exposed on the synthesis top). The Tcl driver sets all of them except `FORWARDING_ENABLE` on the design fileset; for an 80 MHz core clock (the Basys3 default):

```
set repo     <repo_root>
set build    [file join $repo build]
set_property generic \
  "IMEM_WORDS=2048 DMEM_WORDS=2048 \
   IMEM_INIT_FILE=$build/full_core.text.hex \
   DMEM_INIT_FILE=$build/full_core.data.hex \
   CLK_IN_PERIOD_PS=10000 \
   MMCM_MULT=12 MMCM_DIVCLK=1 MMCM_OUT_DIV=15" \
  [current_fileset]
```

7.9 On-chip debugging (ILA)

For FPGA bring-up, Vivado's Integrated Logic Analyzer (ILA) provides a transaction-level view of execution by probing internal nets. The `MARK_DEBUG=true` attribute is already applied in `rtl/top.vhd` to the nets of interest (`pc`, `instr`, `dmem_addr_i`, `dmem_wdata_i`, `dmem_wstrb_i`, `dmem_write_en_i`, `dmem_read_en_i`), so they appear directly in the "Set Up Debug" wizard without further RTL edits.

7.10 Validate a program via the signature window (ILA capture)

The same signature idea used in simulation is used on FPGA: rather than reading BRAM over JTAG, trigger an ILA on stores into the signature window and read the captured store words back from the ILA buffer.

- DMEM base: `0x2000_0000`
- Signature base: `0x2000_0100` (DMEM offset `0x100`)

- Trigger: `dmem_write_en_i == 1` AND `dmem_addr_i == 32'h2000_01XX`
- Capture: `dmem_addr_i` and `dmem_wdata_i`; depth 4096 samples is ample (`full_core.sig` has 10 words).

Hardware Manager Tcl to run the capture and dump the samples to CSV:

```
set ila [lindex [get_hw_ilas] 0]
run_hw_ila    $ila
wait_on_hw_ila $ila
write_hw_ila_data -csv_file signature_capture.csv \
                  [upload_hw_ila_data $ila]
```

Sort the captured rows by `dmem_addr_i` and compare the `dmem_wdata_i` column against `tests/isa/<test>.sig` (one 32-bit word per line).

7.11 Implementation results

Table 1 lists the post-implementation results on ZedBoard (`xc7z020clg484-1`) at the default 80 MHz core clock, measured with Vivado 2024.2, the default implementation strategy, and the `full_core` program. The ILA column is for an ILA on all 134 `MARK_DEBUG` bits, with a depth of 4096 samples and capture control enabled. Other RTL revisions, Vivado versions, devices, or ILA settings give somewhat different numbers.

Published results Papers about MinRV32 report the results of the version that existed when they were written. The FPGA results of the BCI conference paper were obtained with an earlier version in which the core was clocked directly by the 100 MHz board oscillator while the timing constraint specified 80 MHz, so its timing results (WNS +0.112 ns without and +0.269 ns with the ILA) were not checked at the clock that the core actually received. The current version fixes this with the MMCM. Its resource usage is almost the same (1845 against 1842 LUTs without the ILA), its estimated power includes the MMCM (0.261 against 0.162 W without the ILA), and its timing is checked at the real core clock. The `docs/vivado_instructions.md` file lists the differences in detail.

The two BUFGs without the ILA drive the core clock and the MMCM feedback; the ILA adds a third one for the JTAG clock of its debug hub, and it samples on the core clock. The power is the vectorless estimate of Vivado; most of the dynamic power is the MMCM, a fixed cost of the clock generation, and the processor itself accounts for about 0.036 W.

Critical path The slowest paths start at the output of the data-memory block RAM (2.45 ns after the clock edge) and resolve a branch that depends on the loaded value: the IMEM/DMEM read-data mux, the load extension and the Writeback result mux, the forwarding mux of the Execute stage, and the branch comparison produce `PCSrcE`. `PCSrcE` then drives the next-PC adder and the PC register, and `FlushD`/`FlushE`, which reach the synchronous reset inputs of the Fetch/Decode and Decode/Execute registers. Without the ILA the slowest endpoint is such a reset input (11 logic levels), and the PC register is only 0.027 ns behind it (19 logic levels, 9 of them `CARRY4`); with the ILA, the PC register is the slowest endpoint. More than half of the delay is routing, so the slack depends on placement.

Table 1: Post-implementation results on ZedBoard at 80 MHz.

Resource / result	Available	Without ILA	With ILA
Slice LUTs	53200	1845 (3.47%)	3778 (7.10%)
Slice registers (FF)	106400	1523 (1.43%)	4800 (4.51%)
Occupied slices	13300	875 (6.58%)	2053 (15.44%)
F7 muxes	26600	261	280
Block RAM tiles (RAMB36)	140	4 (2.86%)	19 (13.57%)
DSP slices	220	0	0
Bonded IOB	200	3	3
Global clock buffers (BUFG)	32	2	3
MMCM	4	1	1
WNS / WHS (ns)	—	+0.174 / +0.156	+0.240 / +0.065
Total on-chip power (est.)	—	0.261 W	0.289 W
Dynamic	—	0.152 W	0.179 W
of which MMCM	—	0.116 W	0.116 W
Device static	—	0.108 W	0.110 W

MMCM or PLL Each 7-series clock tile has one MMCM and one PLL, and either can make 80 MHz from 100 MHz. With a PLL at the same 1200 MHz internal oscillator (×12/15), the estimated power was slightly higher (0.121 W against 0.116 W) and the design missed timing by 0.178 ns, only because the core logic was placed differently. With a PLL at 800 MHz (×8/10), the total power fell to 0.242 W, but the clock uncertainty grew from 0.069 to 0.082 ns and the WNS fell to +0.050 ns. The power depends on the oscillator frequency and not on the type of block, so the MMCM is kept.

8 Deviations from Harris & Harris

This implementation is based on the Harris & Harris pipelined reference, with intentional changes to keep the RTL modular, more ISA-complete for labs, and synthesis-ready for FPGA BRAM inference.

8.1 Summary table

Area	Deviation (this repository)
ISA coverage	All six RV32I branch types are supported (including unsigned branches), and loads/stores cover byte/half/word widths with sign/zero extension.
Control organization	Control is decoded in D and explicitly registered into E/M/W; signal names are stage-suffixed throughout (`ImmSrcD`, `ALUControlE`, `MemWriteM`, `ResultSrcW`).
Branching	A dedicated branch unit evaluates all branch types instead of relying only on an ALU `Zero` flag.
Datapath flexibility	The ALU A input supports RS1/PC/zero selection (`SrcASelE`) for clean PC-relative and LUI-style flows.
Structural datapath	`datapath_pipe.vhd` uses a structural architecture that instantiates five per-stage wrapper entities (`fetch_stage` through `writeback_stage`), each containing named `pipe_reg` instances and functional-unit instantiations. This gives Vivado's elaborated design view a navigable per-stage hierarchy; the textbook keeps the datapath as a single flat module.
Hazards	A dedicated hazard unit implements forwarding, load-use stalls, and control-hazard flushing; M-stage forwarding is suppressed for loads because load data is only valid in W.
FPGA-ready memories	Instruction and data memories use synchronous reads so the same RTL can infer Block RAM on FPGA tools (Vivado/Quartus). Output registers at the memories act as the relevant pipeline latches (`InstrD` and `ReadDataW`). The instruction memory also has an explicit enable so its registered output can be held stable across decode-stage stalls (textbook-style "freeze InstrD").
Dual-port instruction memory	`imem_dp.vhd` provides two synchronous read ports backed by a single memory array: port A serves instruction fetch while port B serves data-side loads targeting the text region (for `.rodata` constants). This avoids duplicating IMEM in synthesis.
Flush around BRAM outputs	Because BRAM outputs cannot be overridden like a conventional F→D register, a small `flush_d_hold` mechanism inserts a NOP on the following cycle to implement the same logical flush behavior.
Byte-write inference	Stores use a byte-lane write pattern aligned with Xilinx BRAM inference templates (direct per-byte enable assignments).
Explicit read enables	Data memory includes a `MemRead` enable so loads are explicitly qualified rather than being read-always.

Area	Deviation (this repository)
Register file timing	The register file writes on the rising edge (single-edge, BRAM-friendly) and uses a combinational bypass on each read port to preserve same-cycle W→D read-after-write behavior.
Harvard split exposure	Instruction and data ports are split at the top-level wrapper (`top.vhd`) while the internal core wiring remains close to textbook signal naming.

8.2 Hazard-unit design notes (practical guards)

The documentation calls out a few guards that are easy to miss when moving from diagrams to working RTL:

- M-stage forwarding excludes loads (load data is only valid in W).
- RAW stall checks are gated by `RegWrite` to avoid false dependencies on encodings that carry a non-zero `Rd` field but do not write a register.
- A control-flow redirect (taken branch/jump) must override data stalls so the pipeline advances to the correct target and makes forward progress.

8.3 True dual-port IMEM for `.rodata` loads

The core places `.rodata` in instruction memory alongside `.text`, so some loads must read from IMEM (addresses below `DMEM_BASE_ADDR`) and others from DMEM. The top-level wrapper uses `imem_dp` (true dual-port) so instruction fetch and a data-side read can happen simultaneously without duplicating BRAM resources.

Why this matters on FPGA Instantiating two separate single-port IMEMs (one for fetch, one for data-side reads) would infer two independent BRAM blocks in synthesis. A true dual-port memory keeps one shared array:

Configuration	BRAM primitives (example)
Two separate `imem` instances	2×
One `imem_dp` dual-port instance	1×

Scaling (IMEM + DMEM) At 32-bit word width on Xilinx 7-series devices, a `RAMB36E1` stores roughly 1024 words, so BRAM usage scales approximately as `WORDS / 1024` per memory.

- With the default sizing (`IMEM_WORDS=2048`, `DMEM_WORDS=2048`), `imem_dp` is about 2 BRAM36 blocks while `dmem` is about 2.
- Duplicating instruction memory (two separate `imem` instances) would double the IMEM cost (e.g., about 4 BRAM36 blocks for `IMEM_WORDS=2048`).

8.4 Register file: rising-edge write with bypass

The Harris & Harris reference writes the register file on the falling edge so a same-cycle Decode-stage read sees the freshly written value. This repository writes on the rising edge (single-edge clocking for BRAM/timing closure) and adds a simple combinational bypass per read port for the same-cycle W→D case:

```
rs1_data <= rd_data when (rd_write = '1' and rs1_addr = rd_addr and rs1_addr /= "00000")
            else read_reg(regs, rs1_addr);
```

8.5 Textbook vs BRAM-style timing at the stage boundary

Textbook (combinational)	This design (synchronous BRAM)
`InstrF` available same cycle	BRAM delivers `InstrD` on the next rising edge
Separate F→D register for `InstrD`	IMEM output register *is* the F→D latch
Separate M→W register for `ReadDataW`	DMEM output register *is* the M→W latch
Read-always memory	Read enables qualify and/or freeze outputs on stalls

Does this add a pipeline stage? No. Instead of inserting a 6th stage just to "wait for BRAM", the synchronous memory output registers become the stage-boundary latches:

- IMEM: during cycle N, `PCF` is presented as the read address; on the next rising edge (cycle N+1), the BRAM output register updates and the instruction is available as `InstrD` for Decode.
- DMEM (loads): during cycle N, the load address in `ALUResultM` is presented; on the next rising edge (cycle N+1), the BRAM output register updates and the load word is available as `ReadDataW` for Writeback.

The stage names (F/D/E/M/W) describe when values are valid and consumed, not when a BRAM address is presented.

8.6 Why the synchronous-memory change matters

The textbook's combinational-read arrays are convenient for diagrams but do not map cleanly to FPGA Block RAM primitives. This project adopts BRAM-style, clocked reads to keep the educational pipeline *real-hardware* friendly, while preserving the same stage timing and hazard behavior.

9 Quickstart (common flows)

9.1 Run a single test (GHDL)

```
./scripts/run_test.sh full_core
```

9.2 Run a single test with trace and waves

```
./scripts/run_test.sh --trace full_core
./scripts/run_test.sh --fst full_core
./scripts/view_wave.sh build/waves/full_core.fst
```

9.3 Run all tests (regression)

```
./scripts/run_tests.sh
./scripts/run_tests.sh --trace
```

9.4 Generate or check golden signatures (Spike)

```
./scripts/gen_golden.sh --check factorial          # Spike vs tests/isa/factorial.sig
./scripts/gen_golden.sh --words 4 --write my_test  # create tests/isa/my_test.sig
./scripts/check_goldens.sh                         # check every .sig file
```

To run or step through a program on Spike yourself, see Sections 6.10 and 6.11.

9.5 Run a unit testbench (module bring-up)

```
./scripts/run_unit_tb.sh regfile
./scripts/run_unit_tb.sh --fst alu
```

9.6 Build hex images for Vivado/BRAM init

```
./scripts/build_prog.sh tests/isa/full_core.c full_core
ls -1 build/full_core.*   # .text.hex, .data.hex, .elf, ...
cat tests/isa/full_core.sig  # expected signature words (one per line)
```

9.7 Interactive schematic + simulation (DigitalJS, simple module)

```
./scripts/gen_schematic.sh --digitaljs alu
# upload build/schematics/alu.digitaljs.json to https://digitaljs.tilk.eu/
```

9.8 Interactive full-pipeline simulation with firmware loaded

```
./scripts/build_prog.sh tests/isa/full_core.c full_core
./scripts/gen_schematic.sh --digitaljs \
    --mem-init imem_dp=build/full_core.text.hex \
    --mem-init dmem=build/full_core.data.hex \
    top
# upload build/schematics/top.digitaljs.json to https://digitaljs.tilk.eu/
```

9.9 Typical artifacts

- `build/<name>.text.hex`: words for IMEM initialization.
- `build/<name>.data.hex`: words for DMEM initialization.
- `tests/isa/<name>.sig`: expected signature window (tests).
- `build/<name>.elf`: full ELF for symbols/debug.

References

[1] RISC-V International, *The RISC-V Instruction Set Manual: Unprivileged ISA – RV32 Base Integer Instruction Set*, https://docs.riscv.org/reference/isa/unpriv/rv32.html.

[2] Sarah Harris and David Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2021.